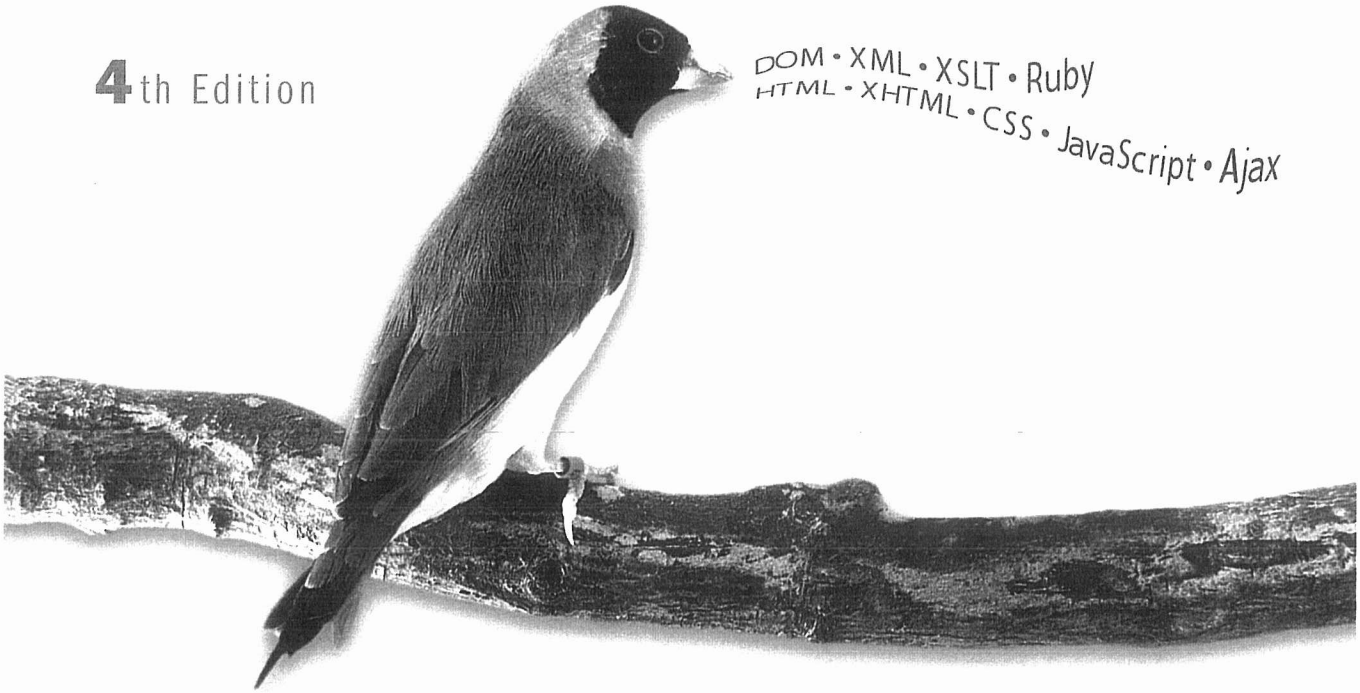
4th Edition

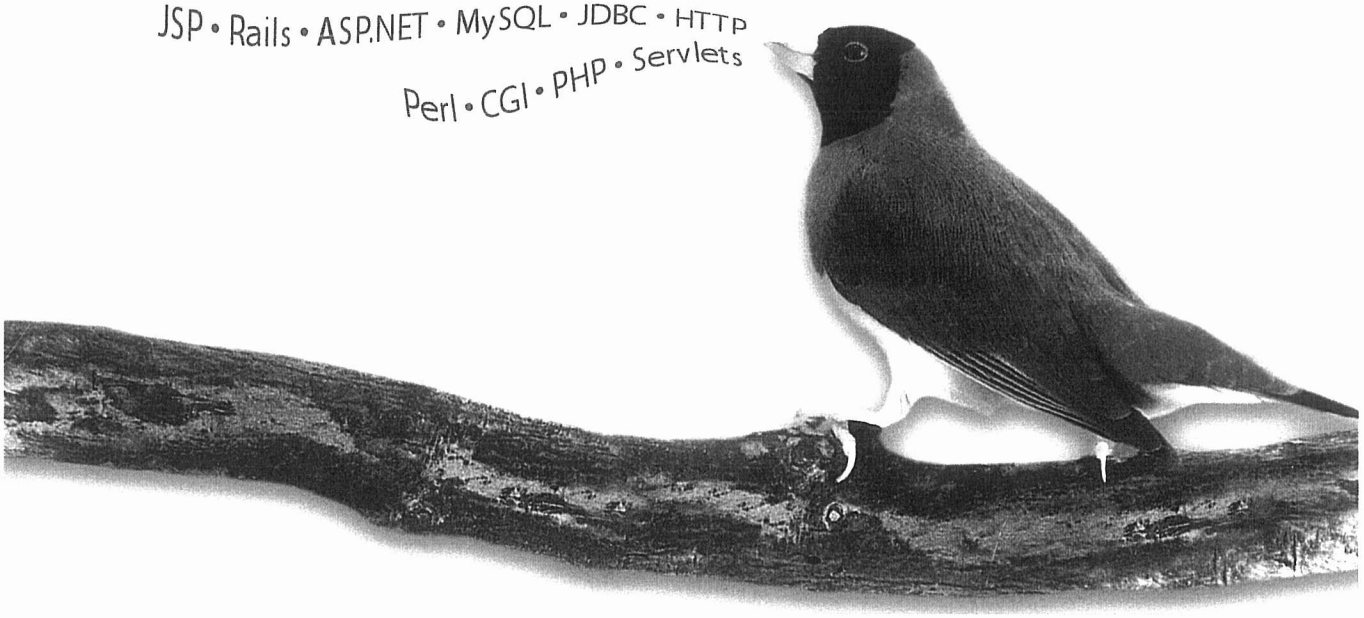
DOM • XML • XSLT • Ruby
HTML • XHTML • CSS • JavaScript • Ajax



Programming the World Wide Web

ROBERT W. SEBESTA

JSP • Rails • ASP.NET • MySQL • JDBC • HTTP
Perl • CGI • PHP • Servlets



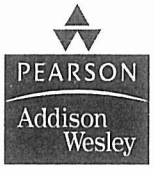
Programming the

World Wide Web

4th Edition

ROBERT W. SEBESTA

University of Colorado at Colorado Springs



Boston San Francisco New York
London Toronto Sydney Tokyo Singapore Madrid
Mexico City Munich Paris Cape Town Hong Kong Montreal

Publisher: Greg Tobin
Acquisitions Editor: Matt Goldstein
Editorial Assistant: Maurene Goo
Associate Managing Editor: Jeffrey Holcomb
Senior Production Supervisor: Marilyn Lloyd
Cover Designer: Joyce Cosentino Wells
Cover Image: © AGE Fotostock America Inc.
Text Design: Gillian Hall
Media Producer: Bethany Tidd
Marketing Assistant: Sarah Milmore
Senior Manufacturing Buyer: Carol Melville
Production Services: The Aardvark Group Publishing Services
Project Manager: Gillian Hall, The Aardvark Group Publishing Services
Copyeditor: Kathleen Cantwell, C4 Technologies
Proofreader: Holly McLean-Aldis
Indexer: Jack Lewis

Access the latest information about Addison-Wesley titles from our World Wide Web site: http:www.aw-bc.com/computing

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in this book, and Addison-Wesley was aware of a trademark claim, the designations have been printed in initial caps or all caps.

The programs and applications presented in this book have been included for their instructional value. They have been tested with care but are not guaranteed for any particular purpose. The publisher does not offer any warranty or representation, nor does it accept any liabilities with respect to the programs or applications.

Library of Congress Cataloging-in-Publication Data

Sebesta, Robert W.
Programming the world wide web / Robert W. Sebesta. -- 4th ed.
p. cm.
Includes bibliographical references and index.
ISBN 978-0-321-48969-2 (alk. paper)
1. Internet programming. 2. World Wide Web. I. Title.
QA76.625.S42 2007
006.7'6--dc22 2007028368

Copyright © 2008 Pearson Education, Inc. All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without the prior written permission of the publisher. Printed in the United States of America. For information on obtaining permission for use of material in this work, please submit a written request to Pearson Education, Inc., Rights and Contracts Department, 501 Boylston Street, Suite 900, Boston, MA 02116, fax your request to 617-671-3447, or e-mail at http://www.pearsoned.com/legal/permissions.htm.

ISBN 13: 9780321489692
ISBN 10: 0-321-48969-1

2 3 4 5 6 7 8 9 10-CRS-10 09 08

To Aidan

Contents

Preface xiii

1 **Fundamentals** 1

1.1 A Brief Introduction to the Internet 2
1.2 The World Wide Web 6
1.3 Web Browsers 7
1.4 Web Servers 8
1.5 Uniform Resource Locators 11
1.6 Multipurpose Internet Mail Extensions 13
1.7 The Hypertext Transfer Protocol 15
1.8 Security 18
1.9 The Web Programmer's Toolbox 20
Summary 28
Review Questions 30
Exercises 32

2 **Introduction to XHTML** 33

2.1 Origins and Evolution of HTML and XHTML 34
2.2 Basic Syntax 36
2.3 Standard XHTML Document Structure 37
2.4 Basic Text Markup 38
2.5 Images 47
2.6 Hypertext Links 52
2.7 Lists 55
2.8 Tables 61
2.9 Forms 69
2.10 Frames 81
2.11 Syntactic Differences between HTML and XHTML 88
Summary 89
Review Questions 90
Exercises 92

3 **Cascading Style Sheets** 95

3.1 Introduction 96
3.2 Levels of Style Sheets 97
3.3 Style Specification Formats 98
3.4 Selector Forms 100
3.5 Property Value Forms 103
3.6 Font Properties 104
3.7 List Properties 110
3.8 Color 114
3.9 Alignment of Text 116
3.10 The Box Model 118
3.11 Background Images 123
3.12 The `<span>` and `<div>` Tags 125
3.13 Conflict Resolution 126

Summary 128
Review Questions 130
Exercises 131

4 **The Basics of JavaScript** 133

4.1 Overview of JavaScript 134
4.2 Object Orientation and JavaScript 137
4.3 General Syntactic Characteristics 138
4.4 Primitives, Operations, and Expressions 141
4.5 Screen Output and Keyboard Input 150
4.6 Control Statements 154
4.7 Object Creation and Modification 162
4.8 Arrays 163
4.9 Functions 168
4.10 An Example 173
4.11 Constructors 174
4.12 Pattern Matching Using Regular Expressions 175
4.13 Another Example 180
4.14 Errors in Scripts 181

Summary 183
Review Questions 185
Exercises 187

5 **JavaScript and XHTML Documents** 189

5.1 The JavaScript Execution Environment 190
5.2 The Document Object Model 191

5.3 Element Access in JavaScript 193
5.4 Events and Event Handling 195
5.5 Handling Events from Body Elements 199
5.6 Handling Events from Button Elements 201
5.7 Handling Events from Text Box and Password Elements 207
5.8 The DOM 2 Event Model 217
5.9 The `navigator` Object 222
5.10 DOM Tree Traversal and Modification 224
Summary 225
Review Questions 226
Exercises 227

6 **Dynamic Documents with JavaScript** 229
6.1 Introduction 230
6.2 Positioning Elements 230
6.3 Moving Elements 236
6.4 Element Visibility 239
6.5 Changing Colors and Fonts 240
6.6 Dynamic Content 244
6.7 Stacking Elements 247
6.8 Locating the Mouse Cursor 250
6.9 Reacting to a Mouse Click 253
6.10 Slow Movement of Elements 254
6.11 Dragging and Dropping Elements 257
Summary 262
Review Questions 262
Exercises 263

7 **Introduction to XML** 265
7.1 Introduction 266
7.2 The Syntax of XML 268
7.3 XML Document Structure 271
7.4 Document Type Definitions 272
7.5 Namespaces 279
7.6 XML Schemas 281
7.7 Displaying Raw XML Documents 289
7.8 Displaying XML Documents with CSS 291
7.9 XSLT Style Sheets 293
7.10 XML Processors 302

7.11 Web Services 304

Summary 305
Review Questions 307
Exercises 309

8 **The Basics of Perl** 311

8.1 Origins and Uses of Perl 312
8.2 Scalars and Their Operations 312
8.3 Assignment Statements and Simple Input and Output 316
8.4 Control Statements 319
8.5 Fundamentals of Arrays 324
8.6 Hashes 329
8.7 References 330
8.8 Functions 331
8.9 Pattern Matching 336
8.10 File Input and Output 339
8.11 An Example 341

Summary 343
Review Questions 345
Exercises 346

9 **Using Perl for CGI Programming** 349

9.1 The Common Gateway Interface 350
9.2 CGI Linkage 351
9.3 Query String Format 354
9.4 The `CGI.pm` Module 356
9.5 A Survey Example 365
9.6 Cookies 377

Summary 381
Review Questions 382
Exercises 383

10 **Servlets and Java Server Pages** 387

10.1 Overview of Servlets 388
10.2 Servlet Details 389
10.3 A Survey Example 392
10.4 Storing Information on Clients 399
10.5 Java Server Pages 412

Summary 426
Review Questions 428
Exercises 429

11 **Introduction to PHP** 431
11.1 Origins and Uses of PHP 432
11.2 Overview of PHP 432
11.3 General Syntactic Characteristics 433
11.4 Primitives, Operations, and Expressions 434
11.5 Output 440
11.6 Control Statements 442
11.7 Arrays 445
11.8 Functions 453
11.9 Pattern Matching 457
11.10 Form Handling 460
11.11 Files 466
11.12 Cookies 469
11.13 Session Tracking 471
Summary 472
Review Questions 474
Exercises 475

12 **Introduction to ASP.NET** 479
12.1 Overview of the .NET Framework 480
12.2 Introduction to C# 483
12.3 Introduction to ASP.NET 491
12.4 ASP.NET Controls 497
12.5 Web Services 513
Summary 517
Review Questions 518
Exercises 520

13 **Database Access through the Web** 521
13.1 Relational Databases 522
13.2 An Introduction to the Structured Query Language 524
13.3 Architectures for Database Access 529
13.4 The MySQL Database System 532
13.5 Database Access with Perl and MySQL 535
13.6 Database Access with PHP and MySQL 539
13.7 Database Access with JDBC and MySQL 548
Summary 556
Review Questions 557
Exercises 559

14 **Introduction to Ruby** 561
14.1 Origins and Uses of Ruby 562
14.2 Scalar Types and Their Operations 562
14.3 Simple Input and Output 570
14.4 Control Statements 573
14.5 Fundamentals of Arrays 578
14.6 Hashes 583
14.7 Methods 585
14.8 Classes 590
14.9 Code Blocks and Iterators 595
14.10 Pattern Matching 597
Summary 601
Review Questions 601
Exercises 602

15 **Introduction to Rails** 605
15.1 Overview of Rails 606
15.2 Document Requests 607
15.3 Processing Forms 615
15.4 Rails Applications with Databases 622
15.5 Layouts 633
Summary 635
Review Questions 636
Exercises 636

16 **Introduction to Ajax** 639
16.1 Overview of Ajax 640
16.2 The Basics of Ajax 642
16.3 Rails with Ajax 651
Summary 659
Review Questions 659
Exercises 660

Appendix A Introduction to Java 661
Appendix B Named Colors 677
Appendix C Java Applets 681

Index 709

Preface

It is difficult to overestimate the effect the World Wide Web has had on the day-to-day lives of people, at least those in the developed countries. In just a few years, we have learned to use the Web for a myriad of disparate tasks, ranging from the mundane task of shopping for airline tickets to the crucial early-morning gathering of business news for a high-stakes day trader.

The speed at which millions of Web sites have appeared would seem to indicate that the technologies used to build them were sitting on the shelf, fully developed and ready to use, even before the Web was developed. Also, one might guess that the tens of thousands of people who built those sites were sitting around unemployed, waiting for an opportunity and already possessing the knowledge and abilities required to carry out this mammoth construction task when it appeared. Neither of these was true. The need for new technologies was quickly filled by a large number of entrepreneurs, some at existing companies and some who started new companies. A large part of the programmer need was filled, at least to the extent to which it has been filled, by new programmers, some straight from high school. Many, however, were previously employed by other sectors of the software development industry. All of them had to learn to use new languages and technologies.

Until recently, programmers learned Web software technologies through company in-house training, a scattering of courses focused on one specific Web technology at colleges and universities, or on their own. A visit to a local bookstore will turn up a large supply of books on those technologies aimed at the practicing professional. In the last few years college courses have begun to appear that attempt to cover a broad spectrum of Web programming technologies. One difficulty encountered by those teaching these courses is the lack of textbooks that are targeted to their needs. Most of the books that discuss Web programming were written for professionals, rather than college students. Such books are typically written to fulfill the needs of professionals, which are quite different from those of college students. One major difference between an academic book and a professional book lies in the assumptions made by the author about the prior knowledge and experience of the audience. The backgrounds of

professionals vary widely, making it difficult to assume much of anything. On the other hand, a book written for junior computer science majors can make some definite assumptions about the background of the reader. This book is written for those students.

The goal of this book is to provide the reader with a comprehensive introduction to the programming tools and skills required to build and maintain server sites on the Web. A wide variety of technologies are used in the construction of a Web site. There are now many books available for professionals that focus on these technologies. For example, there are dozens of books that specifically address only XHTML. The same is true for a half-dozen other technologies. This book provides an overview of how the Web works, as well as descriptions of many of the most widely used Web technologies.

The first three editions of this book were used to teach a junior-level Web programming course at the University of Colorado at Colorado Springs. The challenge for students in the course is to learn to use several different programming languages and technologies in one semester. A heavy load of programming exercises is essential to the success of the course. Students build a basic, static Web site using only XHTML as the first assignment. Throughout the remainder of the semester they add features to their site as the new technologies are discussed in the course. Our students' prior course work in Java, data structures, and assembly language are helpful, as is the fact that many of them know some XHTML before taking the course.

The most important prerequisite to the material of this book is a solid background in programming in some language that supports object-oriented programming. It is helpful to have some knowledge of a second programming language and a bit of UNIX, which helps with the Perl part of the course. Also, UNIX is still a popular platform choice for Web servers. Familiarity with a second language makes learning the new languages easier.

Table of Contents

The book is organized into three parts, introduction (Chapter 1), client-side technologies (Chapters 2–7), and server-side technologies (Chapters 8–16).

Chapter 1 lays the groundwork for the rest of the book. A few fundamentals are introduced, including the history and nature of the Internet, the World Wide Web, browsers, servers, URLs, MIME types, and HTTP. Also included in Chapter 1 are brief overviews of the most important topics of the rest of the book.

Chapter 2 provides an introduction to XHTML, including images, links, lists, tables, frames, and forms. Small examples are used to illustrate the many XHTML elements that are discussed in this chapter.

The topic of Chapter 3 is Cascading Style Sheets, which provide the standard way of imposing style on the content specified in XHTML tags. Because of the size and complexity of the topic, the chapter does not cover all of the aspects

of style sheets. The topics discussed are levels of style sheets, style specification formats, selector formats, property values, and color. Among the properties covered are those for fonts, lists, and margins. Small examples are used to illustrate the subjects being discussed.

Chapter 4 introduces the core of JavaScript, a powerful language that could be used for a variety of different applications. Our interest, of course, is its use in Web programming. Although JavaScript has become a large and complex language, we use the student's knowledge of programming in some other language to leverage our discussion, thereby providing a useful introduction to the language in a manageably small number of pages. Topics covered are the object model of JavaScript, its control statements, objects, arrays, functions, constructors, and pattern matching.

In Chapter 5 we discuss some of the features of JavaScript that are related to XHTML documents. Included is the use of the basic and DOM 2 event and event-handling model, which can be used in conjunction with some of the elements of XHTML documents.

One of the most exciting and interesting applications of JavaScript is for building dynamic XHTML documents using the Document Object Model (DOM). Chapter 6 provides descriptions of a collection of some of the document changes that can be made using JavaScript and the DOM. Included are element positioning, moving elements, changing the visibility of elements, changing the color, style, and size of text, changing the content of tags, changing the stacking order of overlapped elements, slow movement of elements, and dragging and dropping elements.

Chapter 7 presents an introduction to XML, which provides the means to design topic-specific markup languages that can be shared among users with common interests. Included are the syntax and document structure used by XML, data type definitions, namespaces, schemas, and the display of XML documents with both Cascading Style Sheets and XML Transformations. Also included is an introduction to Web services and XML processors.

Chapter 8 introduces the characteristics and capabilities of Perl as a general-purpose programming language. Both before and since the arrival of the Web, the power and flexibility of Perl have been used on a variety of non-Web applications, including UNIX system administration and as the language for many of the small- to medium-size programming tasks formerly done in C. For the most part, we limit our focus on Perl to those parts of the language that are needed for CGI programming. Control statements, arrays, hashes, references, functions, pattern matching, and file input and output are discussed.

Chapter 9 introduces the use of Perl for Common Gateway Interface (CGI) programming. Although there are now alternatives to CGI, it is still widely used, and when it is, it is most often done in Perl. CGI and CGI linkage are introduced first. Then the form of query strings for form data is described. Finally, the `CGI.pm` module is introduced, which greatly simplifies CGI programming in Perl. Several examples are used to show how common CGI tasks are designed and programmed in Perl using `CGI.pm`.

Paul Ammann
George Mason University

Dunren Che
Southern Illinois University

Richard C. Detmer
Middle Tennessee State University

Tony Gauvin
University of Maine at Fort Kent

Jonathan Geisler
Taylor University

David Green
University of Alabama at Birmingham

George Holmes
University of Arkansas

Arpad Kelemen
Niagara University

Linda Kenney
University of New Hampshire

Kadathur Lakshmanan
State University of New York at Brockport

Greg Madey
University of Notre Dame

Tim Margush
University of Akron

Robert Noonan
College of William and Mary

Jeff Offutt
George Mason University

James Richards
Towson University

Earl M. Rodd
Malone College

Bennett Setzer
Kennesaw State University

Chen-chi Shing
Radford University

Marius C. Silaghi
Florida Institute of Technology

Donald S. Szarkowicz
Indiana University Northwest

Michael Weiss
Carleton University

Jonwook Woo
California State University, Los Angeles

Tom Wulf
University of Cincinnati

Mir Farook Ali

Elizabeth Leboffee

Pete DePasquale, The College of New Jersey; Jonathan Geisler, Taylor University; and Martin Stepp, University of Washington served as technical reviewers for the fourth edition.

Matt Goldstein, Editor; Maurene Goo, Editorial Assistant; Marilyn Lloyd, Production Manager; and Sarah Milmore, Marketing Assistant, all deserve my gratitude for their encouragement and help in completing the manuscript. Also, thanks to Gillian Hall at The Aardvark Group for quickly and accurately converting the collection of files I provided into a bound book.

CHAPTER

1

Fundamentals

1.1 A Brief Introduction to the Internet
1.2 The World Wide Web
1.3 Web Browsers
1.4 Web Servers
1.5 Uniform Resource Locators
1.6 Multipurpose Internet Mail Extensions
1.7 The Hypertext Transfer Protocol
1.8 Security
1.9 The Web Programmer's Toolbox
Summary • *Review Questions* • *Exercises*

The lives of most inhabitants of industrialized countries, as well as some in unindustrialized countries, have been changed forever by the advent of the World Wide Web. Although this has had some downsides—for example, easier access to pornography and gambling and the ease with which those with destructive ideas can propagate those ideas to others—on balance, the changes have been enormously positive. Many of us use the Internet and the World Wide Web daily, communicating with friends, relatives, and business associates through e-mail, shopping for virtually anything that can be purchased anywhere, and digging up a limitless variety and amount of information, from movie theater schedules to hotel room prices in cities halfway around the world to the history and characteristics of the culture of some small and obscure society of humans. Constructing the software and data that provide all of this information requires knowledge of several different technologies, such as markup languages and meta-markup languages, as well as programming skills in a

myriad of different programming languages, some specific to the World Wide Web and some designed for general-purpose computing. This book is meant to provide the required background and a basis for acquiring the knowledge and skills necessary to build the World Wide Web sites that provide both the information users want and the advertising that helps pay for its presentation.

This chapter lays the groundwork for the remainder of the book. It begins with introductions to and some history of the Internet and the World Wide Web. Then it discusses the purposes and some of the characteristics of Web browsers and servers. Next, it describes uniform resource locators (URLs), which specify addresses for resources available on the Web. Following this, it introduces Multipurpose Internet Mail Extensions, which provide ways in which file types can be specified—and which are required because of the many different formats in which information can be represented in files. Next, it discusses the Hypertext Transfer Protocol (HTTP), which provides the communication interface for connections between browsers and Web servers. Finally, the chapter provides brief overviews of some of the tools commonly used by Web programmers, including XHTML, XML, JavaScript, Java, Perl, PHP, Ruby, Rails, and Ajax. All of these are discussed in far more detail in the remainder of the book (XHTML in Chapters 2 and 3; JavaScript in Chapters 4, 5, and 6; XML in Chapter 7; Perl in Chapters 8 and 9; PHP in Chapter 11; Java in Chapter 10 and Appendices A and C; Ruby in Chapters 14 and 15; Rails in Chapter 15; and Ajax in Chapter 16).

1.1 A Brief Introduction to the Internet

Virtually every topic discussed in this book is related to the Internet. Therefore, we begin with a quick introduction to the Internet itself.

1.1.1 Origins

In the 1960s the U.S. Department of Defense (DoD) became interested in developing a new large-scale computer network in the 1960s. The purposes of this network were communications, program sharing, and remote computer access for researchers working on defense-related contracts. One fundamental requirement was that the network be sufficiently robust so that even if some network nodes were lost due to sabotage, war, or some more benign reason, the network could continue to function. The DoD's Advanced Research Projects Agency (ARPA)[1] funded the construction of the first such network, which connected about a dozen ARPA-funded research laboratories and universities. The first node of this network was established at UCLA in 1969.

Because it was funded by ARPA, the network was named ARPAnet. Despite the initial intentions, the primary early use of ARPAnet was simple text-based

1. ARPA was renamed Defense Advanced Research Projects Agency (DARPA) in 1972.

communications through e-mail. Because ARPAnet was available only to laboratories and universities that conducted ARPA-funded research, the great majority of educational institutions were not connected. As a result, a number of other networks were developed during the late 1970s and early 1980s, with BITNET and CSNET among them. BITNET, which is an acronym for Because It's Time Network, began at the City University of New York. It was built initially to provide electronic mail and file transfers. CSNET, which is an acronym for Computer Science Network, connected the University of Delaware, Purdue University, the University of Wisconsin, RAND Corporation, and Bolt, Beranek, and Newman. Its initial purpose was to provide electronic mail. For a variety of reasons, neither BITNET nor CSNET became a dominant national network.

A new national network, NSFnet, was created in 1986. It was sponsored, of course, by the National Science Foundation (NSF). NSFnet initially connected the NSF-funded supercomputer centers at five universities. Soon after being established, it became available to other academic institutions and research laboratories. By 1990, NSFnet had replaced ARPAnet for most nonmilitary uses, and a wide variety of organizations had established nodes on this network—by 1992, NFSnet connected more than 1 million computers around the world. In 1995 a small part of NSFnet returned to being a research network. The rest became known as the Internet, although this term was used much earlier for both ARPAnet and NSFnet.

1.1.2 What the Internet Is

The Internet is a huge collection of computers connected in a communications network. These computers are of every imaginable size, configuration, and manufacturer. In fact, some of the devices connected to the Internet—such as plotters and printers—are not computers at all. The innovation that allows all of these diverse devices to communicate with each other is a single, low-level protocol, the Transmission Control Protocol/Internet Protocol (TCP/IP). TCP/IP became the standard for computer network connections in 1982, and it can be used directly to allow a program on one computer to communicate with a program on another computer via the Internet. In most cases, however, a higher-level protocol runs on top of TCP/IP. Nevertheless, it's important to know that TCP/IP provides the low-level interface that allows all computers (and other devices) connected to the Internet to appear exactly the same.

Rather than connecting every computer on the Internet directly to every other computer on the Internet, normally the individual computers in an organization are connected to each other in a local network. One node on this local network is physically connected to the Internet. So, the Internet is actually a network of networks rather than a network of computers.

Obviously, all devices connected to the Internet must be uniquely identifiable.

1.1.3 Internet Protocol Addresses

For people, Internet nodes are identified by names; for computers, they are identified by numeric addresses. This exactly parallels the relationship between a variable name in a program, which is for people, and the variable's numeric memory address, which is for the machine.

The Internet Protocol (IP) address of a machine connected to the Internet is a unique 32-bit number. IP addresses usually are written (and thought of) as four 8-bit numbers, separated by periods. The four parts are separately used by Internet-routing computers to decide where a message must go next to get to its destination.

Organizations are assigned blocks of IPs, which they in turn assign to their machines that need Internet access—which now includes most computers. For example, a small organization may be assigned 256 IP addresses, such as `191.57.126.0` to `191.57.126.255`. Very large organizations, such as the Department of Defense, may be assigned 16 million IP addresses, which include IP addresses with one particular first 8-bit number, such as `12.0.0.0` to `12.255.255.255`.

Although people nearly always type domain names into their browsers, the IP works just as well. For example, the IP for United Airlines (`www.ual.com`) is `209.87.113.93`. So, if a browser is pointed at `http://209.87.113.93`, it will be connected to the United Airlines Web site.

In late 1998 a new IP standard, IPv6, was approved, although it has not yet been widely implemented. The most significant change was to expand the address size from 32 bits to 128 bits. This is a change that will soon be essential because the number of remaining unused IP addresses is diminishing rapidly. This new standard can be found at `ftp://ftp.isi.edu/in-notes/rfc2460.txt`.

1.1.4 Domain Names

Because people have difficulty dealing with and remembering numbers, machines on the Internet also have textual names. These names begin with the name of the host machine, followed by progressively larger enclosing collections of machines, called *domains*. There may be two, three, or more domain names. The first domain name, which appears immediately to the right of the hostname, is the domain of which the host is a part. The second domain name gives the domain of which the first domain is a part. The last domain name identifies the type of organization in which the host resides, which is the largest domain in the site's name. For organizations in the United States, `edu` is the extension for educational institutions, `com` specifies a company, `gov` is used for the U.S. government, and `org` is used for many other kinds of organizations. In other countries, the largest domain is often an abbreviation for the country—for example, `se` is used for Sweden, and `kz` is used for Kazakhstan.

Consider this sample address:

```
movies.comedy.marxbros.com
```

Here, `movies` is the hostname and `comedy` is `movies`'s local domain, which is a part of `marxbros`'s domain, which is a part of the `com` domain. The hostname and all of the domain names are together called a *fully qualified domain name*.

Because IP addresses are the addresses used internally by the Internet, the fully qualified domain name of the destination for a message, which is what is given by a browser user, must be converted to an IP address before the message can be transmitted on the Internet to the destination. These conversions are done by software systems called *name servers*, which implement the Domain Name System (DNS). Name servers serve a collection of machines on the Internet and are operated by organizations that are responsible for the part of the Internet to which those machines are connected. All document requests from browsers are routed to the nearest name server. If the name server can convert the fully qualified domain name to an IP address, it does so. If it cannot, the name server sends the fully qualified domain name to another name server for conversion. Like IP addresses, fully qualified domain names must be unique. Figure 1.1 shows how fully qualified domain names requested by a browser are translated into IPs before they are routed to the appropriate Web server.

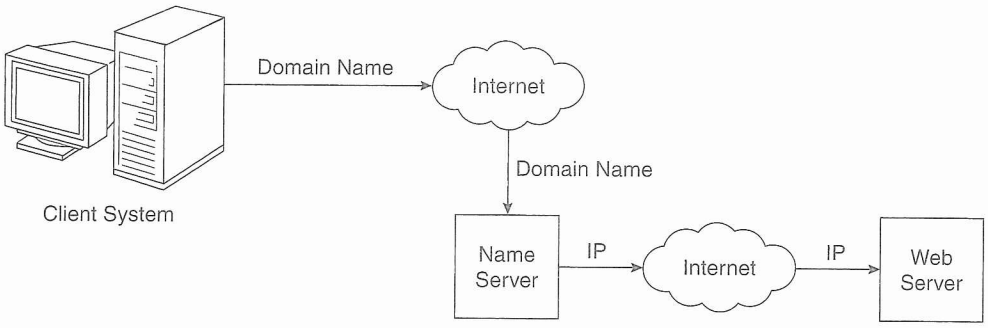


Figure 1.1 Domain name conversion

One way to determine the IP address of a Web site is by using `telnet` on the fully qualified domain name. This is illustrated in Section 1.7.1.

By the mid-1980s, a collection of different protocols that run on top of TCP/IP had been developed to support a variety of Internet uses. Among these, the most common were `telnet`, which was developed to allow a user on one computer on the Internet to log on to and use another computer on the Internet; File Transfer Protocol (`ftp`), which was developed to transfer files among computers on the Internet; Usenet, which was developed to serve as an electronic bulletin board; and `mailto`, which was developed to allow messages to be sent from the user of one computer on the Internet to other users on other computers on the Internet.

This variety of protocols, each with its own user interface and useful for only the purpose for which it was designed, restricted the growth of the Internet. Users were required to learn all the different interfaces to gain all the

advantages of the Internet. Before long, however, a better approach developed: the World Wide Web.

1.2 The World Wide Web

Programming for the World Wide Web is the topic of this book, so a brief overview of the Web is certainly in order here.

1.2.1 Origins

In 1989 a small group of people led by Tim Berners-Lee at CERN (Conseil European pour la Recherce Nucleaire, or the European Laboratory for Particle Physics) proposed a new protocol for the Internet as well as a system of document access to use it.[2] The intent of this new system, which the group named the World Wide Web, was to allow scientists around the world to use the Internet to exchange documents describing their work.

The proposed new system was designed to allow a user anywhere on the Internet to search for and retrieve documents from databases on any number of different document-serving computers. By late 1990, the basic ideas for the new system had been fully developed and implemented on a NeXT computer at CERN. In 1991, the system was ported to other computer platforms and released to the rest of the world.

For the form of its documents, the system used *hypertext*, which is text with embedded links to text in other documents to allow nonsequential browsing of textual material. The idea of hypertext had been developed earlier and had appeared in Xerox's NoteCards and Apple's HyperCard in the mid-1980s.

From here on, we will refer to the World Wide Web simply as "the Web." The units of information on the Web have been referred to by several different names; among them, the most common are *pages*, *documents*, and *resources*. Perhaps the best of these is *documents*, although that seems to imply only text. *Pages* is widely used, but it is misleading in that Web units of information often have more than one of the kind of pages that make up printed media. There is some merit to calling these units *resources* because that covers the possibility of nontextual information. This book will use *documents* and *pages* more or less interchangeably, but we prefer *documents* in most situations.

Documents are sometimes just text, usually with embedded links to other documents, but they often also include images, sound recordings, or other kinds of media. When a document contains nontextual information, it is called *hypermedia*.

In an abstract sense, the Web is merely a vast collection of documents, some of which are connected by links. These documents are accessed by Web browsers, introduced in Section 1.3, and are provided by Web servers, introduced in Section 1.4.

---

2. Although Berners-Lee's college degree (from Oxford) was in physics, his first stint at CERN was as a consulting software engineer. Berners-Lee was born and raised in London.

1.2.2 Web or Internet?

It is important to understand that the Internet and the Web are not the same thing. The *Internet* is a collection of computers and other devices connected by equipment that allows them to communicate with each other. The *Web* is a collection of software and protocols that has been installed on most, if not all, of the computers on the Internet. The Internet was quite useful before the Web was developed, and it is still useful without it. However, it is now the case that most users of the Internet use it through the Web.

1.3 Web Browsers

When two computers communicate over some network, in many cases one acts as a client and the other as a server. The client initiates the communication, which is often a request for information stored on the server, which sends that information back to the client. The Web, as well as many other systems, operates in this client/server configuration.

Documents provided by servers on the Web are requested by *browsers*, which are programs running on client machines. They are called browsers because they allow the user to browse the resources available on servers. The first browsers were text based—they were not capable of displaying any sort of graphic information, nor did they have a graphical user interface. This effectively constrained growth of Web use. In early 1993, this changed with the release of Mosaic, the first browser with a graphical user interface. Mosaic was developed at the National Center for Supercomputer Applications (NCSA) at the University of Illinois. This interface provided convenient access to the Web for users who were neither scientists nor software developers. The first release of Mosaic ran on UNIX systems using the X Window system. By late 1993, versions of Mosaic for Apple Macintosh and Microsoft Windows systems had been released. Finally, users of the computers connected to the Internet around the world had a powerful way to access anything on the Web anywhere in the world. The result of this power and convenience was explosive growth in Web usage.

A browser is a client on the Web because it initiates the communication with a server, which waits for a request from a client before doing anything. In the simplest case, a browser requests a static document from a server. The server locates the document and sends it to the browser, which displays it for the user. However, more complicated situations are common. For example, the server may provide a document that requests input from the user through the browser. After the user supplies the requested input, it is transmitted from the browser to the server, which may perform some computation using it and then return a new document to the browser to inform the user of the results of the computation. Sometimes a browser directly requests the execution of a program stored on the server. The output of the program is then returned to the browser.

Although the Web supports a variety of protocols, the most common one is the Hypertext Transfer Protocol (HTTP). HTTP provides a standard form of

communications between browsers and Web servers. Section 1.7 provides an introduction to HTTP.

The most commonly used browsers are Microsoft Internet Explorer (IE), which runs only on PCs that use one of the Microsoft Windows operating systems,[3] and Firefox, which is available in versions for several different computing platforms, including Windows, Mac OS, and Linux. There are several other browsers available, such as the close relatives of Firefox, Mozilla Suite, and Netscape Navigator, as well as Opera and Apple's Safari. However, because the great majority of browsers now in use are IE or Firefox, in this book we focus on those two.

1.4 Web Servers

Web servers are programs that provide documents to requesting browsers. Servers are slave programs: They act only when requests are made to them by browsers running on other computers on the Internet.

The most commonly used Web servers are Apache, which has been implemented for a variety of computer platforms, and Microsoft's Internet Information Server (IIS), which runs under Windows operating systems. As of March 2007, there were over 110 million Web hosts in operation, 59 percent of which were Apache, about 31 percent of which were IIS, and the remainder were spread thinly over a large number of others. (The third-place server was SunONE, with just over 1.7 percent.)[4]

1.4.1 Web Server Operation

Although having clients and servers is a natural consequence of information distribution, this configuration offers some additional benefits for the Web. Serving information does not take a great deal of time. On the other hand, displaying information on client screens is time consuming. Because Web servers need not be involved in this display process, they can handle many clients. So, it is both a natural and an efficient division of labor to have a small number of servers provide documents to a large number of clients.

Web browsers initiate network communications with servers by sending them URLs (discussed in Section 1.5). A URL can specify one of two different things: the address of a data file stored on the server that is to be sent to the client, or a program stored on the server that the client wants executed, with the output of the program returned to the client.

---

3. Actually, versions 4 and 5 of IE (IE4 and IE5) were also available for Macintosh computers, and IE4 was available for UNIX systems. However, IE6 and IE7 are currently available for Windows platforms only.

4. These statistics are from `http://www.netcraft.com`.

All the communications between a Web client and a Web server use the standard Web protocol, Hypertext Transfer Protocol (HTTP), which is discussed in Section 1.7.[5]

When a Web server begins execution, it informs the operating system under which it is running that it is now ready to accept incoming network connections through a specific port on the machine. While in this running state, the server runs as a background process in the operating system environment. A Web client, or browser, opens a network connection to a Web server, sends information requests and possibly data to the server, receives information from the server, and closes the connection. Of course, other machines exist between browsers and servers on the network—specifically, network routers and domain-name servers. This section, however, focuses on just one part of Web communication: the server.

Simply put, the primary task of a Web server is to monitor a communications port on its host machine, accept HTTP commands through that port, and perform the operations specified by the commands. All HTTP commands include a URL, which includes the specification of a host machine. When the URL is received, it is translated into either a filename (in which case the file is returned to the requesting client) or a program name (in which case the program is run and its output is sent to the requesting client). This sounds pretty simple, but as is the case in many other simple-sounding processes, a large number of complicating details are involved.

All current Web servers have a common ancestry: the first two servers, developed at CERN in Europe and NCSA at the University of Illinois. Currently, the most common server configuration is Apache running on some version of UNIX.

1.4.2 General Server Characteristics

Most available servers share common characteristics, regardless of their origin or the platform on which they run. This section provides brief descriptions of some of these characteristics.

The file structure of a Web server has two separate directories. The root of one of these is called the *document root*. The file hierarchy that grows from the document root stores the Web documents to which the server has direct access and normally serves to clients. The root of the other directory is called the *server root*. This directory, along with its descendant directories, stores the server and its support software.

The files stored directly in the document root are those available to clients through top-level URLs. Typically, clients do not access the document root directly in URLs; rather, the server maps requested URLs to the document root, whose location is not known to clients. For example, suppose that the site name is `www.bloomers.com`, which happens to be a UNIX-based system. Further suppose that the document root is named `topdocs` and is stored in the

5. Some of these communications use the secure version of HTTP, HTTPS.

`/admin/web` directory, making its address `/admin/web/topdocs`. A request for a file from a client with the URL `http://www.bloomers.com/petunias.html` will cause the server to search for the file with the file path `/admin/web/topdocs/petunias.html`. Likewise, the URL `http://www.bloomers.com/bulbs/tulips.html` will cause the server to search for the file with the address `/admin/web/topdocs/bulbs/tulips.html`.

Many servers allow part of the servable document collection to be stored outside the directory at the document root. The secondary areas from which documents can be served are called *virtual document trees*. For example, the original configuration of a server might store all its servable documents from the primary system disk on the server machine. Later, the collection of servable documents might outgrow that disk, in which case part of the collection could be stored on a secondary disk. This secondary disk might reside on the server machine or on some other machine on a local area network. To support this arrangement, the server is configured to direct request URLs with a particular file path to a storage area separate from the document-root directory. Sometimes files of different types of content, such as images, are stored outside the document root.

Early servers provided few services other than the basic process of returning requested files or the output of programs whose execution had been requested. The list of additional services has grown steadily in the past few years. Contemporary servers are large and complex systems that provide a wide variety of client services. Many servers can support more than one site on a computer, potentially reducing the cost of each site and making their maintenance more convenient. Such secondary hosts are called *virtual hosts*.

Some servers can serve documents that are in the document root of other machines on the Web; in this case, they are called *proxy servers*.

Although Web servers were originally designed to support only the HTTP protocol, many now support `ftp`, Gopher, News, and mail. In addition, nearly all Web servers can interact with database systems through Common Gateway Interface (CGI) programs and server-side scripts.

1.4.3 Apache

Apache began as the NCSA server, `httpd`, with some added features. The name Apache has nothing to do with the Native American tribe of the same name. Rather, it came from the nature of its first version, which was *a patch*y version of the `httpd` server. As seen in the usage statistics given at the beginning of this section, Apache is by far the most widely used Web server. The primary reasons for this are as follows: It is an excellent server because it's both fast and reliable. Furthermore, it is open-source software, which means it is free and is managed by a large team of volunteers, a process that efficiently and effectively maintains the system. Finally, it is one of the best available servers for Unix-based systems, which are the most popular for Web servers.

Apache is capable of providing a long list of services beyond the basic process of serving documents to clients. It takes an entire book to describe all these

services. It also takes a long list of words to explain how an Apache server can be configured to the tastes of the site manager. Only a tiny glimpse of how the actions of Apache can be customized is given here.

When Apache begins execution, it reads its configuration information from a file and sets its parameters to operate accordingly. A new copy of Apache includes default configuration information for "typical" operation. The site manager modifies this configuration information to fit his or her particular needs and tastes.

For historical reasons, there are three configuration files in an Apache server: `httpd.conf`, `srm.conf`, and `access.conf`. Only one of these, `httpd.conf`, actually stores the directives that control an Apache server's behavior. The other two point to `httpd.conf`. This file contains the list of directives that specify the server's operation. These directives are described at `http://httpd.apache.org/docs/2.2/mod/quickreference.html`.

1.4.4 IIS

Although Apache has been ported to the Windows platforms, it is not the most popular server on those systems. Because the Microsoft IIS server is supplied as part of Windows—and because it is a reasonably good server—most Windows-based Web servers use IIS. Apache and IIS provide similar varieties of services.

From the point of view of the site manager, the most important difference between Apache and IIS is that Apache is controlled by a configuration file that is edited by the manager to change Apache's behavior. With IIS, server behavior is modified by changes made through a window-based management program, named the IIS snap-in, which controls both IIS and `ftp`. This program allows the site manager to set parameters for the server.

Under Windows XP, the IIS snap-in is accessed by going to Control Panel, Administrative Tools, and IIS Admin. Clicking on this last selection takes you to a window that allows starting, stopping, or pausing IIS. This same window allows IIS parameters to be changed when the server has been stopped.

1.5 Uniform Resource Locators

Uniform (or universal)[6] resource locators (URLs) are used to identify documents (resources) on the Internet. There are many different kinds of resources, identified by different forms of URLs.

1.5.1 URL Formats

All URLs have the same general format:

scheme : object-address

6. Fortunately, resource addresses are usually referred to as URLs, so whether it is *uniform* or *universal* is usually irrelevant.

The scheme is often a communications protocol. Common schemes include `http`, `ftp`, `gopher`, `telnet`, `file`, `mailto`, and `news`. Different schemes use object addresses that have differing forms. Our main interest is in the HTTP protocol, which supports the Web. It is used to request and send eXtensible Hypertext Markup Language (XHTML) documents. In the case of HTTP, the form of the object address of a URL is as follows:

`//`fully-qualified-domain-name`/`path-to-document

Another scheme of interest to us is `file`. The `file` protocol means that the document resides on the machine running the browser. This is useful for testing documents to be made available on the Web, without actually making them visible to any other browser. When `file` is the protocol, the fully qualified domain name is omitted, making the form of such URLs as follows:

`file://`path-to-document

Because we focus on XHTML documents in this book, we limit the remainder of our discussion of URLs to the HTTP protocol.

The hostname is the name of the server computer that stores the document (or provides access to it on some other computer). Messages to a host machine must be directed to the appropriate process running on the host for handling. Such processes are identified by their associated port numbers. The default port number of Web server processes is 80. If a server has been configured to use some other port number, it is necessary to attach that port number to the hostname in the URL. For example, if the Web server is configured to use port 800, the hostname must have `:800` attached.

URLs can never have embedded spaces.[7] Also, there is a collection of special characters that cannot appear in a URL, including semicolons, colons, and ampersands (`&`). To include a space or one of the disallowed special characters in a URL, the character must be coded as a percent sign (`%`) followed by the two-digit hexadecimal ASCII code for the character. For example, if `San Jose` is a domain name, it must be typed as `San%20Jose` (`20` is the hexadecimal ASCII code for a space). All of the details of URLs can be found at `http://www.w3.org/Addressing/URL/URI_Overview.html`.

1.5.2 URL Paths

The path to the document for the HTTP protocol is similar to a path to a file or directory in the file system of an operating system: a sequence of directory names and a filename, all separated by whatever separator character the operating system uses. For UNIX servers, the path is specified with forward slashes; for Windows servers, it is specified with backward slashes. Most browsers allow the user to specify the separators incorrectly—for example, using forward

7. Actually, some browsers incorrectly accept spaces in URLs, although this is nonstandard behavior.

slashes in a path to a document file on a Windows server. For example, for UNIX, you might have the following:

```
http://www.gumboco.com/files/f99/storefront.html
```

The path in a URL can differ from a path to a file because a URL need not include all directories on the path. A path that includes all directories along the way is called a *complete path*. In most cases, the path to the document is relative to some base path that is specified in the configuration files of the server. Such paths are called *partial paths*. For example, if the server's configuration specifies that the root directory for files it can serve is `files/f99`, the previous URL is specified as follows:

```
http://www.gumboco.com/storefront.html
```

If the specified document is a directory rather than a single document, the directory's name is followed immediately by a slash, as in the following:

```
http://www.gumboco.com/departments/
```

Sometimes a directory is specified (with the trailing slash) but its name is not given, as in the following example:

```
http://www.gumboco.com/
```

The server then searches at the top level of the directory in which servable documents are normally stored for something it recognizes as a home page. By convention, this is often a file named `index.html`. The home page usually includes links that allow the user to find the other related servable files on the server.

If the directory does not have a file that the server recognizes as being a home page, a directory listing is constructed and returned to the browser.

1.6 Multipurpose Internet Mail Extensions

A browser needs some way of determining the format of a document it receives from a Web server. Without knowing the form of a document, the browser would be unable to render it. The forms of these documents are specified with the Multipurpose Internet Mail Extensions (MIME).

1.6.1 Type Specifications

MIME was developed to allow different kinds of documents to be sent using Internet mail. These could be various kinds of text, video data, or sound data. Because the Web has similar needs, MIME was adopted as the way to specify document types transmitted over the Web. A Web server attaches a MIME format specification to the beginning of the document that it is about to provide to a browser. When the browser receives the document from a Web server, it uses the included MIME format specification to determine what to do with the doc-

ument. If the content is text, for example, the MIME code tells the browser that it is text and also indicates the particular kind of text it is. If the content is sound, the MIME code tells the browser that it is sound and then gives the particular representation of sound so that the browser can choose a program to which it has access to produce the transmitted sound.

MIME specifications have the following form:

type/subtype

The most common MIME types are `text`, `image`, and `video`. The most common text subtypes are `plain` and `html`. The most common image subtypes are `gif` and `jpeg`. The most common video subtypes are `mpeg` and `quicktime`. A list of MIME specifications is stored in the configuration files of every Web server. In the remainder of this book, when we say *document type*, we mean both the document's type and its subtype.

Servers determine the type of a document by using the filename's extension as the key into a table of types. For example, the extension `.html` tells the server that it should attach `text/html` to the document before sending it to the requesting browser.[8]

Browsers also maintain a conversion table for looking up the type of a document by its filename extension. However, this is used only when the server does not specify a MIME type, which may be the case for some older servers. In all other cases, the browser gets the document type from the MIME header provided by the server.

1.6.2 Experimental Document Types

Experimental subtypes are sometimes used. The name of an experimental subtype begins with `x-`, as in `video/x-msvideo`. Any Web provider can add an experimental subtype by having its name added to the list of MIME specifications stored in the Web provider's server. For example, a Web provider might have a handcrafted database whose contents he or she wants to make available to others through the Web. Of course, this raises the issue of how the browser can display the database. As you might expect, the Web provider must supply a program that the browser can call when it needs to display the contents of the database. These programs either are external to the browser, in which case they are called *helper applications*, or are code modules that are inserted into the browser, in which case they are called *plug-ins*.

Every browser has a set of MIME specifications it can handle. All can deal with `text/plain` (unformatted text) and `text/html` (HTML files), among others. It sometimes occurs that a particular browser cannot handle some specific document type, even though the type is widely used. These cases are handled in the same way as the experimental types described previously. The browser determines the helper application or plug-in it needs by examining the

8. This is not necessarily correct. XHTML documents also use the `.html` file extension, but strictly speaking should use a different MIME type.

browser configuration file, which provides an association between file types and their required helpers or plug-ins. If the browser does not have an application or a plug-in that it needs to render a document, an error message is displayed.

A browser can indicate to the server the document types it prefers to receive, as discussed in Section 1.7.

1.7 The Hypertext Transfer Protocol

All Web communications transactions use the same protocol—the Hypertext Transfer Protocol (HTTP). The current version of HTTP is 1.1. It is formally defined as RFC 2616, which was approved in June 1999. RFC 2616 is available at the Web site for the World Wide Web Consortium (W3C), `http://www.w3.org`. This section provides a brief introduction to HTTP.

HTTP consists of two phases, the request and the response. Each HTTP communication (request or response) between a browser and a Web server consists of two parts, a header and a body. The header contains information about the communication; the body contains the data of the communication, if there is any.

1.7.1 The Request Phase

The general form of an HTTP request is as follows:

1. HTTP method Domain part of the URL HTTP version
2. Header fields
3. Blank line
4. Message body

The following is an example of the first line of an HTTP request:

```
GET  /storefront.html  HTTP/1.1
```

Only a few request methods are defined by HTTP, and even a smaller number of these are typically used. Table 1.1 lists the most commonly used methods.

Table 1.1 HTTP Request Methods

Method	Description
`GET`	Returns the contents of the specified document
`HEAD`	Returns the header information for the specified document
`POST`	Executes the specified document, using the enclosed data
`PUT`	Replaces the specified document with the enclosed data
`DELETE`	Deletes the specified document

Among the methods given in Table 1.1, `GET` and `POST` are the most frequently used. `POST` was originally designed for tasks such as posting a news article to a newsgroup. Its most common use now is to send form data to the server, along with a request to execute a program on the server that will process the form data.

Following the first line of an HTTP communication is any number of header fields, most of which are optional. The format of a header field is the field name followed by a colon and the value of the field. There are four categories of header fields:

1. *General:* For general information, such as the date
2. *Request:* Included in request headers
3. *Response:* For response headers
4. *Entity:* Used in both request and response headers

One common request field is the `Accept` field, which specifies a preference of the browser for the MIME type of the requested document. More than one `Accept` field can be specified if the browser is willing to accept documents in more than one format. For example:

```
Accept: text/plain
Accept: text/html
Accept: image/gif
```

A wildcard character, the asterisk (`*`), can be used to specify that part of a MIME type can be anything. For example, if any kind of text is acceptable, the `Accept` field could be as follows:

```
Accept: text/*
```

The `Host:` *`hostname`* request field gives the name of the host. The `Host` field is required for HTTP 1.1. The `If-Modified-Since:` *`date`* request field specifies that the requested file should be sent only if it has been modified since the given date.

If the request has a body, the length of that body must be given with a `Content-length` field, which gives the length of the response body in bytes. `POST` method requests require this field because they send data to the server.

The header of a request must be followed by a blank line, which is used to separate the header from the body of the request. Requests that use the `GET`, `HEAD`, and `DELETE` methods do not have bodies. In these cases, the blank line signals the end of the request.

A browser is not necessary to communicate with a Web server; `telnet` also can be used. Consider the following command, given at the command line of any widely used operating system:

```
> telnet blanca.uccs.edu http
```

This command creates a connection to the `http` port on the `blanca.uccs.edu` server. The server responds with the following:[9]

```
Trying 128.198.162.60 ...
Connected to blanca
Escape character is '^]'.
```

The connection to the server is now complete, and HTTP commands can be given. For example:

```
GET /respond.html HTTP/1.1
Host: blanca.uccs.edu
```

The header of the response to this request is given in Section 1.7.2.

1.7.2 The Response Phase

The general form of an HTTP response is as follows:

1. Status line
2. Response header fields
3. Blank line
4. Response body

The status line includes the HTTP version used, a three-digit status code for the response, and a short textual explanation of the status code. For example, most responses begin with the following:

```
HTTP/1.1  200  OK
```

The status codes begin with `1`, `2`, `3`, `4`, or `5`. The general meanings of the five categories specified by these first digits are shown in Table 1.2.

Table 1.2 First Digits of HTTP Status Codes

First Digit	Category
1	Informational
2	Success
3	Redirection
4	Client error
5	Server error

9. Notice that this `telnet` request returns the IP of the server.

One of the more common status codes is one users never want to see: `404 Not Found`, which means the requested file could not be found. Of course, `200 OK` is what users want to see, because it means the request was handled without error. The `500` code means the server has encountered a problem and was not able to fulfill the request.

After the status line, the server sends a response header, which can contain several lines of information about the response, each in the form of a field. The only essential field of the header is `Content-type`.

The following is the response header for the request given near the end of Section 1.7.1:

```
HTTP/1.1  200  OK
Date: Tues, 18 May 2004 16:45:13 GMT
Server: Apache (Red-Hat/Linux)
Last-modified: Tues, 18 May 2004 16:38:38 GMT
ETag: "1f1223-16c-92dc9f80"
Accept-ranges: bytes
Content-length: 364
Connection: close
Content-type: text/html, charset=ISO-8859-1
```

The response header must be followed by a blank line, as is the case for request headers. The response data follows the blank line. In the preceding example, the response body would be the html file, `respond.html`.

In HTTP versions prior to 1.1, when a server finished sending a response to the client, the communications connection was closed. However, the default operation of HTTP 1.1 is that the connection is kept open for a time so that a client can make several requests over a short period of time without needing to reestablish the communications connection with the server. This change led to significant increases in the efficiency of the Web.

1.8 Security

It does not take a great deal of contemplation to realize that the Internet and the Web are fertile grounds for security problems. On the Web server side, anyone on the planet with a computer, a browser, and an Internet connection can request the execution of software on the server computer. He or she can also access data and databases stored on the server computer. On the browser end, the problem is similar: any server to which the browser points can download software that is to be executed on the browser host machine. Such software potentially can access parts of the memory and memory devices attached to that machine that are not related to the needs of the original browser request. In effect, on both ends, it is like allowing any number of total strangers into your house and preventing them from leaving anything in the house, taking anything from the house, or altering anything in the house. The larger and more complex the design of the house, the more difficult it will be to prevent any of those activities. The same is true for Web servers and browsers: The larger and more

complex they are, the more difficult it is to prevent security breaches. Today's browsers and Web servers are indeed large and complex software systems, so security is a significant problem in Web applications.

The subject of Internet and Web security is extensive and complicated, so much so that more than a few books that discuss it have been written. Therefore, in this one section of one chapter of one book, there can be no more than a brief sketch of some of the subtopics of security.

One of the aspects of Web security is the matter of getting one's data from the browser to the server and having the server deliver data back to the browser without anyone or any device intercepting or corrupting that data along the way. Consider just the simplest case, that of transmitting a credit card number to a company from which a purchase is being made. The security issues for this transaction are as follows:

1. *Privacy*—it must not be possible for the credit card number to be stolen while on its way to the company's server.
2. *Integrity*—it must not be possible for the credit card number to be modified on its way to the company's server.
3. *Authentication*—it must be possible for both the purchaser and the seller to be certain of each other's identity.
4. *Nonrepudiation*—it must be possible to legally prove that the message was actually sent and received.

The basic tool to support privacy and integrity is encryption. Data to be transmitted is converted, or encrypted into a different form, which is virtually impossible to decrypt for someone (or some computer) who is not supposed to access the data. So, if data is intercepted while en route between Internet nodes, the interceptor cannot use the data because he or she cannot decrypt it. Both encryption and decryption are done with a key and a process (applying the key to the data). Encryption is not a process first developed for the Internet. Julius Caesar used a crude encryption process on the messages he sent to his field generals while at war. Until the middle 1970s, the process used the same key for both encryption and decryption. Because both the sender and receiver used the same key, the initial problem was how to transmit the key from the sender to the receiver.

This problem was solved in 1976 by Whitfield Diffie and Martin Hellman of Stanford University, who developed public-key encryption. In **public-key encryption**, a public key and a private key are used, the public key to encrypt messages and the private key to decrypt messages. A communicator, say Joe, has an inversely related pair of keys, one public and one private. The public key can be distributed to all organizations that might send Joe messages. All of them can use the public key to encrypt messages to Joe, who can decrypt the messages with his matching private key. This works because the private key need never be transmitted, and also because it is virtually impossible to compute the private key from its public key. The technical wording for this is that it is "computationally infeasible" to determine the private key from its public key.

The most widely used public-key algorithm is named RSA, developed in 1977 by three MIT professors, Ron Rivest, Adi Shamir, and Leonard Adleman, the first letters of whose last names were used to name the algorithm. Most large companies now use RSA for e-commerce.

Another completely different security problem for the Web is the intentional and malicious destruction of data on computers attached to the Internet. The number of different ways this can be done has increased steadily over the lifespan of the Web. Their sheer numbers have also grown rapidly. There is now a continuous stream of new and increasingly devious denial-of-service (DoS), viruses, and worms being discovered, which have caused billions of dollars of damage, primarily to businesses that use the Web heavily. Of course, huge damage also has been done to home computer systems through Web intrusions.

DoS attacks can be created simply by flooding a Web server with requests, overwhelming its ability to operate correctly. Most DoS attacks are conducted using networks of virally infected "zombie" computers, whose owners are unaware of their sinister use. So, DoS and viruses are often related.

Viruses are programs that often arrive in a system in attachments to e-mail messages or attached to free downloaded programs. Then they attach to other programs. When executed, they replicate themselves and can themselves overwrite memory, attached memory devices, destroying programs and data alike. Two viruses that were extensively destructive appeared in 2000 and 2001, the ILOVEYOU virus and the CodeRed virus, respectively.

Worms damage memory, like viruses, but spread on their own, rather than being attached to other files. Perhaps the most famous worm so far has been the Blaster worm, launched in 2003.

DoS, virus, and worm attacks are created by malicious people referred to as *hackers*. The incentive for these people apparently is simply the feeling of pride and accomplishment they derive from being able to cause huge amounts of damage by outwitting the designers of Web software systems.

Protection against viruses and worms is provided by antivirus software, which must be updated frequently so that it can detect and protect against the continuous stream of new viruses and worms.

1.9 The Web Programmer's Toolbox

This section provides an overview of the most common tools used in Web programming—some are programming languages, but some are not. The tools discussed are XHTML, a markup language; XML, a meta-markup language; JavaScript, Java, Perl, PHP, and Ruby, which are programming languages; AJAX, which is a Web technology that uses JavaScript and XML; Rails, which is a development framework for Web-based database access systems; and a few high-level markup document editing systems, which are neither. Web programs and scripts are divided into two categories, client side and server side, according to where they are interpreted or executed. XHTML and XML are client-side

languages; Perl, PHP, and Ruby are server-side languages; JavaScript is most often a client-side language, although it can be used for both; and Java has been used for both.

We begin with the most basic tool, XHTML.

1.9.1 Overview of XHTML

At the onset, it is important to realize that XHTML is not a programming language. It cannot be used to describe computations. Its purpose is to describe the general form and layout of documents to be displayed by a browser.

The word *markup* comes from the publishing world, where it is used to describe what production people do with a manuscript to specify to a printer how the text, graphics, and other elements in the book should appear in printed form. XHTML is not the first markup language used with computers. TeX and LaTeX are older markup languages for use with digital text; they are now used primarily to specify how mathematical expressions and formulas should appear in print.

An XHTML document is a mixture of content and controls. The controls are specified by the tags of XHTML. The name of a tag specifies the category of its content. Most XHTML tags consist of a pair of syntactic markers that are used to delimit particular kinds of content. The pair of tags and their content together are called an *element*. For example, a paragraph element specifies that its content, which appears between its opening tag, `<p>`, and its closing tag, `</p>`, is a paragraph. A browser has a default style (font, font style, font size, and so forth) for paragraphs, which is used to display the content of a paragraph element.

Some tags include attribute specifications that provide some additional information for the browser. In the following example, the attribute specifies the location of its image content:

```
<img src = "redhead.jpg"/>
```

In this case, the image document stored in `redhead.jpg` is to be displayed at the position in the document in which this tag appears.

XHTML 1.0 was introduced in early 2000 by the W3C as an alternative to HTML 4.01, which was at that time (and still is) the latest version of HTML. XHTML 1.0 is nothing more than HTML 4.01 with stronger syntactic rules. These stronger rules are those of XML (see Section 1.8.4). The current version, XHTML 1.1, was released in May 2001 as a replacement for XHTML 1.0. Chapter 2, "Introduction to XHTML," provides a description of a large subset of XHTML.

1.9.2 Tools for Creating XHTML Documents

XHTML documents can be created with a general purpose text editor. There are two kinds of tools that can simplify this task: XHTML editors and what-you-see-is-what-you-get (WYSIWYG, pronounced *wizzy-wig*) XHTML editors.

XHTML editors provide shortcuts to producing repetitious tags such as those used to create the rows of a table. They also may provide a spell-checker and a syntax-checker, and they may color code the XHTML in the display to make it easier to read and edit.

A more powerful tool for creating XHTML documents is a WYSIWYG XHTML editor. Using a WYSIWYG XHTML editor, the writer can see the document that the XHTML describes while writing the XHTML. WYSIWYG XHTML editors are very useful for beginners who want to create simple documents without learning XHTML and for users who want to prototype the appearance of a document. On the other hand, these editors sometimes produce poor-quality XHTML. In some cases, they create proprietary tags that some browsers will not recognize.

Three examples of WYSIWYG XHTML editors are Microsoft FrontPage, Macromedia Dreamweaver, and Adobe PageMill. All three allow the user to create XHTML-described documents without requiring the user to know XHTML. They cannot handle all of the tags of XHTML, but they are very useful for creating many of the common features of documents. Among these three, FrontPage is by far the most widely used. Information on PageMill is available at `http://www.adobe.com/`, information on Dreamweaver is available at `http://www.macromedia.com/`, and information on FrontPage is available at `http://www.microsoft.com/frontpage/`.

1.9.3 Plug-ins and Filters

Two different kinds of converters can be used to create XHTML documents. *Plug-ins* are programs that can be integrated with a word processor. Plug-ins add new capabilities to the word processor, such as toolbar buttons and menu elements that provide convenient ways to insert XHTML into the document being created or edited. After such insertions, the document is displayed using the XHTML. So, the plug-in makes the word processor appear to be an XHTML editor that provides WYSIWYG XHTML document development. The end result of this process is an XHTML document. The plug-in also makes available all the tools that are inherent to the word processor during XHTML document creation, such as a spell-checker and a thesaurus.

A second kind of converter is a *filter*, which converts an existing document in some form, such as LaTeX or Microsoft Word, to XHTML. Filters are never part of the editor or word processor that created the document. This is an advantage because they can be platform-independent. For example, a WordPerfect user working on a Macintosh computer can provide documents that can be later converted to XHTML using a filter running on a UNIX platform. The disadvantage of filters is that creating XHTML documents with a filter is a two-step process: First you create the document and then you use a filter to convert it to XHTML.

Neither plugs-ins nor filters produce XHTML documents that, when displayed by browsers, have the identical appearance of that produced by the word processor.

The two advantages of both plug-ins and filters, however, are that existing documents produced with word processors can be easily converted to XHTML and that users can produce XHTML documents using a word processor with which they are familiar. This obviates the need to learn to format text using XHTML directly. For example, once you learn to create tables with your word processor, it is easier to use that process than to learn to define tables directly in XHTML.

The XHTML output produced by both filters and plug-ins often must be modified, usually using a simple text editor, to perfect the appearance of the displayed document on the browser. Because this new XHTML file cannot be converted back to its original form (regardless of how it was created), you will have two different source files for a document. This inevitably leads to version problems during maintenance of the document. This is clearly a disadvantage of using converters.

1.9.4 Overview of XML

HTML is defined using the Standard Generalized Markup Language (SGML), which is a language for defining markup languages (such languages are called meta-markup languages). XML (eXtensible Markup Language) is a simplified version of SGML, designed to allow users to easily create markup languages that fit their own needs. XHTML is defined using XML. Whereas XHTML users must use the predefined set of tags and attributes, when a user creates his or her own markup language using XML, the set of tags and attributes are designed for the application at hand. For example, if a group of users wants a markup language to describe data about weather phenomena, that language could have tags for cloud forms, thunderstorms, and low-pressure centers. The content of these tags would be restricted to relevant data. If such data is described using XHTML, cloud forms could be put in paragraphs, but then they could not be distinguished from thunderstorm elements, which would also be paragraphs.

Whereas XHTML describes the overall layout and some presentation hints for general information, XML-based markup languages describe data and its meaning through their individualized tags and attributes. XML does not specify any presentation details.

The great advantage of XML is that application programs can be written to use the meanings of the tags in the given markup language to find specific kinds of data and process it accordingly. The syntax rules of XML, along with the syntax rules for a specific XML-based markup language, allow documents to be validated before any application attempts to process their data. This means that all documents that use a specific markup language can be checked to determine whether they are in the standard form for such documents. This greatly simplifies the development of application programs that process the data in XML documents.

1.9.5 Overview of JavaScript

JavaScript is a client-side scripting language whose primary uses in Web programming are to validate form data and to create dynamic XHTML documents.

The name JavaScript is misleading because the relationship between Java and JavaScript is tenuous, except for some of the syntax. One of the most important differences between JavaScript and most common programming languages is that JavaScript is dynamically typed. This is virtually the opposite of strongly typed languages such as C++ and Java.

JavaScript "programs" are usually embedded in XHTML documents.[10] These XHTML documents are downloaded when they are requested by browsers. The JavaScript code in an XHTML document is interpreted by the browser on the client.

One of the most important applications of JavaScript is to dynamically create and modify documents. JavaScript defines an object hierarchy that matches a hierarchical model of an XHTML document. Elements of an XHTML document are accessed through these objects, providing the basis for dynamic documents.

Chapter 4, "The Basics of JavaScript," provides a more detailed look at JavaScript. Chapter 5, "JavaScript and XHTML Documents," and Chapter 6, "Dynamic Documents with JavaScript," discuss the use of JavaScript to provide access to and dynamic modification of XHTML documents.

1.9.6 Overview of Java

Java is based on C++ but differs from that language in several important ways. First, Java is much smaller than C++ and is considerably safer, particularly in the areas of pointers and array index range errors. Although simpler than C++, Java is not a simple language by any measure. Appendix A provides a quick introduction to Java, focusing on the parts of the language most often used for writing applets and servlets.

Java was designed by Sun Microsystems, which controls its continuing development. It was originally developed to program household appliances. As always, timing is exquisitely important: Java was still in development when Web usage began to explode. Java's designers realized that Java would be a powerful way to provide a computational capability for XHTML documents. Rather than running programs on the Web server to provide a computational capability, a special kind of Java program called an *applet* can be resident on the server, but a compiled version of the applet can be downloaded to the browser when requested by the XHTML document being displayed by the browser.

Compiled Java is represented in an idealized machine language called *bytecodes*. When the browser receives a bytecode applet, it "executes" it using a Java virtual machine, which is an interpreter for bytecodes. Java applets are discussed in

10. We quote the word *programs* to indicate that these are not programs in the general sense of the self-contained collections of C++ or C code we normally call programs.

Appendix C, "Java Applets." The use of applets moves these computations from servers, which are often very busy, to the browser, which often has little to do. This is similar to how JavaScript works. The primary differences are that JavaScript code is physically part of an XHTML document, whereas applets are stored separately from the XHTML. Moreover, JavaScript is less powerful than Java.

There are many computational tasks in a Web interaction that must occur on the server, such as processing order forms and accessing server-resident databases. A form of Java class called a *servlet* can be used for these applications.

Java can also be used as a server-side scripting language. An XHTML document with embedded Java scriptlets is one form of Java Server Pages. Both servlets and Java Server Pages are discussed in Chapter 10, "Servlets and Java Server Pages."

Microsoft also provides a system for embedding programming code in an XHTML document, this time in any of a variety of programming languages. This system is named Active Server Pages (ASP.NET). ASP.NET is discussed in Chapter 13, "Introduction to ASP.NET."

1.9.7 Overview of Perl

Before applets and embedded scripts were developed, a computational capability was provided for XHTML documents by allowing the document to request the execution of virtually any program on the server. This is done using the Common Gateway Interface (CGI). Briefly, CGI is a standard way in which a browser and a server communicate to run a program on the server and return the output of that program to the browser.

CGI programs can be written in any programming language that is supported by the server machine. The most commonly used language for CGI programs is Perl. Perl dominates the CGI programming business because it is highly portable and because it has several capabilities that are needed for this kind of programming. No other programming language fits the application as well.

The features of Perl that make it ideal for CGI programming are its direct access to operating system functions, its powerful character string pattern-matching operations, and its ability to include database operations. Perl is a very expressive and powerful language whose use is not limited to CGI programming. In recent years, it has replaced C for many small to medium applications. Although Perl began as a UNIX language, it has been ported to all common computer systems, including PCs running Windows and Apple Macintosh systems.

Perl's syntax is similar to that of C. However, Perl code is not compiled into machine language and executed, as is C. Instead, it is compiled to an intermediate language and interpreted.

The subset of Perl that is most useful for CGI programming is introduced in Chapter 8, "The Basics of Perl." Its use in CGI programming is discussed in Chapter 9, "Using Perl for CGI Programming."

1.9.8 Overview of PHP

PHP is a server-side scripting language specifically designed for Web applications (unlike Perl, which is a general-purpose language that happens to be good for server-side software). PHP code is embedded in XHTML documents, as is the case with JavaScript. With PHP, however, the code is interpreted on the server before the XHTML document is delivered to the requesting client. A requested document that includes PHP code is preprocessed to interpret the PHP code and insert its output into the XHTML document. The browser never sees PHP code and is not aware that a requested document originally included PHP code.

PHP is similar to JavaScript, both in terms of its syntactic appearance and in terms of the dynamic nature of its strings and arrays. Both JavaScript and PHP use dynamic data typing, meaning that the type of a variable is controlled by the most recent assignment to it. PHP's arrays are a combination of Perl's arrays and Perl's hashes (associative arrays). The language includes a large number of predefined functions for manipulating arrays.

PHP allows simple access to XHTML form data, so form processing is easy with PHP. PHP provides support for many different database management systems. This makes it an excellent language for building programs that need Web access to databases.

1.9.9 Overview of Ruby

Ruby (Thomas, et al., 2005) is an object-oriented interpretive scripting language designed by Yukihiro Matsumoto (a.k.a. Matz) in the early 1990s and released in 1996. Since then it has continually evolved and its level of usage has grown rapidly. The original motivation for Ruby was dissatisfaction of its designer with the earlier languages, Perl and Python.

The primary characterizing feature of Ruby is that it is a pure object-oriented language, just as is Smalltalk. Every data value is an object and all operations are via method calls. The operators in Ruby are only syntactic mechanisms to specify method calls for the corresponding operations. Because they are methods, many of the operators can be redefined by user programs. All classes, whether predefined or user-defined, can be subclassed.

Both classes and objects in Ruby are dynamic in the sense that methods can be dynamically added to either. This means that both classes and objects can have different sets of methods at different times during execution. So, different instantiations of the same class can behave differently.

The syntax of Ruby is related to that of Eiffel and Ada. There is no need to declare variables, because dynamic typing is used. In fact, all variables are references and do not have types, though the objects they reference do.

Our interest in Ruby is based on Ruby's use with the Web development framework, Rails (see Section 1.9.10). Rails was designed for use with Ruby, and it is Ruby's primary use in Web programming. Programming in Ruby is introduced in Chapter 14, "Introduction to Ruby."

Ruby is culturally interesting because it is the first programming language designed in Japan that has achieved relatively widespread use.

1.9.10 Overview of Rails

Rails is a development framework for Web-based applications that access databases. A framework is a system in which much of the more-or-less standard software parts are furnished by the framework, so they need not be written by the applications developer. ASP.NET is also a development framework for Web-based applications. Rails, because of its intimate connection with Ruby, is often called Ruby on Rails, or simply RoR. Rails was developed by David Heinemeier Hansson in the early 2000s and was released to the public in July 2004. Since then, it has rapidly gained widespread interest and usage. Rails is based on the Model-View-Controller (MVC) architecture for applications, which clearly separates the presentation and the data model from program logic.

Rails applications are tightly bound to relational databases. Many Web applications are closely integrated with database access, so this is a widely applicable architecture.

Rails can and often is used in conjunction with Ajax. Rails uses the JavaScript framework Prototype to support Ajax and interactions with the JavaScript model of the document being displayed by the browser. Rails also provides other support for developing Ajax, including producing visual effects.

Rails was designed to be used with Ruby and makes use of the strengths of that language. Furthermore, Rails is written in Ruby. Using Rails is introduced in Chapter 15, "Introduction to Rails."

1.9.11 Overview of Ajax

Ajax, shorthand for *A*synchronous *Ja*vaScript + *X*ML, has been around for a few years, but did not acquire its catchy name until 2005.[11] The idea of Ajax is relatively simple, but results in a different way of viewing and building Web interactions. This new approach results in an enriched Web experience for those using a certain category of Web interactions.

In a traditional (as opposed to Ajax) Web interaction, the client sends messages to the server, either through clicking a link in the document being displayed on the browser or by submitting forms to the server. After the link has been clicked or the form has been submitted, the client waits until the server responds with a new document. The entire browser display is then replaced by the new document. Complicated documents take a significant amount of time to be transmitted from the server to the client and more time to be rendered by the browser. In Web activities that are relatively intense and remain active for a sig-

11. Ajax was named by Jesse James Garrett, who has on numerous occasions stated that Ajax is shorthand, not an acronym. Thus, we spell it Ajax, not AJAX.

nificant amount of time, the delay for receiving and rendering a complete response document can be disruptive to the user.

In an Ajax Web application, there are two variations from the traditional Web interaction. First, the communication from the browser to the server is asynchronous; that is, the browser need not wait for the server to respond, the browser user can continue whatever he or she was doing while the server finds and transmits the requested document and the browser renders the new document. Second, the document provided by the server usually is only a relatively small part of the displayed document, and therefore it takes less time to be transmitted and rendered. These two changes can result in much faster interactions between the browser and the server.

The x in Ajax, XML, is there because in many cases the data supplied by the server is in the form of an XML document, which provides the new data to be placed in the displayed document. However, in some cases it is plain text, which may be either data or even JavaScript code.

The goal of Ajax is to have Web-based applications become closer to desktop (client resident) applications, in terms of the speed of interactions and thereby the user experience. Wouldn't we all like our Web-based applications to be as responsive as our word processors?

Ajax has some advantages over the competing technologies of ASP.NET and JSP. First and foremost, the technologies that support Ajax are already resident in nearly all Web browsers and servers. This is in contrast to both ASP.NET and JSP, which still have far-from-complete coverage. Second, using Ajax does not require the learning of a new tool or language. Rather, it requires only a new way of thinking about Web interactions.

Ajax is discussed in more depth in Chapter 16, "Introduction to Ajax."

Summary

The Internet began in the late 1960s as the ARPAnet, which was eventually replaced by NSFnet for nonmilitary users. NSFnet later became known as the Internet. There are now millions of computers around the world connected to the Internet. Although much of the network control equipment is different and many kinds of computers are connected, all of these connections are made through the TCP/IP protocol, making them all appear, at least at the lowest level, the same to the network.

Two kinds of addresses are used on the Internet: IP addresses for computers, which are four-part numbers; and fully qualified domain names for people, which are words separated by periods. Fully qualified domain names are translated to IP addresses by name servers running DNS. A number of different information interchange protocols have been created, including `telnet`, `ftp`, and `mailto`.

The Web began in the late 1980s at CERN as a means for physicists to efficiently share the results of their work with colleagues at other locations. The

fundamental idea of the Web is to transfer hypertext documents among computers using the HTTP protocol on the Internet.

Browsers request XHTML documents from Web servers and display them for users. Web servers find and send requested documents to browsers. All documents are addressed on the Internet using URLs; the specific protocol to be used is the first field of the URL. URLs also include the fully qualified domain name and a file path to the specific document on the server. The type of a document that is delivered by a Web server appears in the first line of the document as a MIME specification. Web sites can create their own experimental MIME types, provided that they also furnish a program that allows the browser to present the document's contents to the user.

HTTP is the standard protocol for Web communications. HTTP requests are sent on the Internet from browsers to Web servers; HTTP responses are sent from Web servers to browsers to fulfill those requests. The most commonly used HTTP requests are `GET` and `POST`, both of which require URLs.

Web programmers use several languages to create the documents that servers can provide to browsers. The most basic of these is XHTML, the standard markup language for describing how Web documents should be presented by browsers. Tools that can be used without specific knowledge of XHTML are available to create XHTML documents. A plug-in is a program that can be integrated with a word processor to make it possible to use the word processor to create XHTML. A filter converts a document written in some other format to XHTML. XML is a meta-markup language that provides a standard way to define new markup languages.

JavaScript is a client-side scripting language that can be embedded in XHTML to describe simple computations. JavaScript code is interpreted by the browser on the client machine; it provides access to the elements of an XHTML document, as well as the ability to change those elements dynamically.

Java is a modern, widely used, general-purpose programming language. The two most interesting constructs provided by Java for Web programmers are applets and servlets. Applets reside on the Web server but can be requested by browsers, at which time a compiled version of the applet is transferred to the browser, which interprets the applet. This provides another means of specifying computations for Web documents that take place on the browser. Servlets are server-side Java programs that can be used for form processing and database access.

Perl is a programming language that is often used as a server-side vehicle to describe computations upon request from browsers. Interactions between browsers and Perl programs on the server are done through CGI. PHP is a server-side, XHTML-embedded scripting language. Its uses are similar to those of CGI programs. Ajax is an approach that uses JavaScript to create more responsive interactions between a browser and a server, using asynchronous requests for partial documents that do not require the browser to replace the current displayed document completely.

Ruby is a relatively recent object-oriented scripting language that is introduced here primarily because of its use in Rails, a Web applications framework.

Rails provides a significant part of the code required to build Web applications that access databases, allowing the developer to spend his or her time on the specifics of the application without the drudgery of dealing with all of the housekeeping details.

Review Questions

1.1 What was one of the fundamental requirements for the new national computer network proposed by the DoD in the 1960s?

1.2 What protocol is used by all computer connections to the Internet?

1.3 What is the form of an IP address?

1.4 Describe a fully qualified domain name.

1.5 What is the task of a DNS name server?

1.6 What is the purpose of `telnet`?

1.7 In the first proposal for the Web, what form of information was to be interchanged?

1.8 What is hypertext?

1.9 What category of browser, introduced in 1993, led to a huge expansion in use of the Web?

1.10 In what common situation is the document returned by a Web server created after the request is received?

1.11 What is the document root of a Web server?

1.12 What is a virtual document tree?

1.13 What is the server root of a Web server?

1.14 What is a virtual host?

1.15 What is a proxy server?

1.16 What does the `file` protocol specify?

1.17 How do partial paths to documents work in Web servers?

1.18 When a browser requests a directory without giving its name, what is the name of the file that is normally returned by the Web server?

1.19 What is the purpose of a MIME type specification in a request/response transaction between a browser and a server?

1.20 What must a Web server furnish the browser when it returns a document with an experimental MIME type?

1.21 Describe the purposes of the five most commonly used HTTP methods.

1.22 What is the purpose of the `Accept` field in an HTTP request?

1.23 What response header field is most often required?

1.24 Prior to HTTP 1.1, how long were connections between browsers and servers normally maintained?

1.25 What important capability is lacking in a markup language?

1.26 What problem is addressed by using a public key approach to encryption?

1.27 Is it practically possible to compute the private key associated with a given public key?

1.28 What is the difference between a virus and a worm?

1.29 What appears to motivate a hacker to create and disseminate a virus?

1.30 What is a plug-in?

1.31 What is a filter XHTML converter?

1.32 Why must code generated by a filter often be modified by hand before use?

1.33 What is the great advantage of XML over XHTML for describing data?

1.34 How many different tags are predefined in an XML-based markup language?

1.35 What is the relationship between Java and JavaScript?

1.36 Where is JavaScript code interpreted?

1.37 Where are Java applets interpreted?

1.38 Where are Java servlets interpreted?

1.39 What is the purpose of the Common Gateway Interface?

1.40 What features of Perl make it ideal for CGI programming?

1.41 Where are CGI programs executed (or interpreted)?

1.42 Where is PHP code interpreted?

1.43 In what ways is PHP similar to JavaScript?

1.44 Which programming languages are used in Ajax applications?

1.45 In what fundamental way does an Ajax Web application differ from a traditional Web application?

1.46 In what ways is Ruby more object-oriented than Java?

1.47 In what country was Ruby developed?

1.48 When was Rails first released?

1.49 For what particular kind of Web application was Rails designed?

1.50 What is a software development framework?

Exercises

1.1 For the following products, what brand do you have access to, what is its version number, and what is the latest available version?

a. Browser
b. Web server
c. Perl
d. Java
e. PHP

1.2 Search the Web for information on the history of the following technologies and write a brief overview of those histories.

a. TCP/IP
b. SGML
c. XHTML
d. ARPAnet
e. BITNET
f. XML
g. Rails
h. Ajax

CHAPTER

2

Introduction to XHTML

2.1 Origins and Evolution of HTML and XHTML
2.2 Basic Syntax
2.3 Standard XHTML Document Structure
2.4 Basic Text Markup
2.5 Images
2.6 Hypertext Links
2.7 Lists
2.8 Tables
2.9 Forms
2.10 Frames
2.11 Syntactic Differences between HTML and XHTML
Summary • *Review Questions* • *Exercises*

This chapter introduces the most commonly used subset of the eXtensible Hypertext Markup Language (XHTML). Because of the simplicity of XHTML, the discussion moves quickly. The chapter begins with a brief history of the evolution of HTML and XHTML, followed by a description of the form of tags and the structure of an XHTML document. Then tags used to specify the presentation of text are discussed, including those for line breaks, paragraph breaks, headings, and block quotations, as well as tags for specifying the style and relative size of fonts. This is followed by a description of the formats and uses of images in Web documents. Next, hypertext links are introduced. Three kinds of

lists—ordered, unordered, and definition—are then covered. After that, the XHTML tags and attributes used to specify tables are discussed. The next section of the chapter introduces forms, which provide the means to collect information from Web clients. The following section discusses frames, which provide a way to divide the browser window into smaller rectangles, each of which can display a different document. Finally, the last section describes the syntactic differences between HTML and XHTML.

2.1 Origins and Evolution of HTML and XHTML

HTML is defined using the Standard Generalized Markup Language (SGML), which is an International Standards Organization (ISO) standard notation for describing text-formatting languages.[1] The original intent of HTML was different from those of other text-formatting languages, which dictate all of the presentation details of text, such as font style, size, and color. Rather, HTML was designed to specify document structure at a higher and more abstract level, necessary because HTML-specified documents had to be displayable on a variety of computer systems, often using different browsers.

The addition of style sheets to HTML in the late 1990s advanced its capabilities closer to those of other text-formatting languages by providing a way to include the specification of presentation details. These are introduced in Chapter 3, "Cascading Style Sheets."

2.1.1 Versions of HTML and XHTML

The original version of HTML was designed, in conjunction with the structure of the Web and the first browser, at Conseil European pour la Recherce Nucleaire or European Laboratory for Particle Physics (CERN). Use of the Web began its meteoric rise in 1993 with the release of MOSAIC, the first graphical Web browser. Not long after MOSAIC was commercialized and marketed by Netscape, Microsoft began developing its browser, Internet Explorer (IE). The release of IE marked the beginning of a four-year marketing competition between Netscape and Microsoft. During this time, both companies worked feverously to develop their own extensions to HTML in an attempt to gain market advantage. This naturally led to incompatible versions of HTML, both between the two developers and also between older and newer releases within the same company. All of these differences made it a serious challenge to Web content providers to design HTML documents that could be viewed by the different browsers.

In late 1994 Berners-Lee started the World Wide Web Consortium (W3C), which had as one of its primary purposes to develop and distribute standards for Web technologies, starting with HTML. The first HTML standard,

1. Not all text-formatting languages are based on SGML; for example, PostScript and LaTeX are not.

HTML 2.0, was released in 1995. It was followed by HTML 3.2 in early 1997. Up to this point, W3C was playing catch up, and HTML 3.2 was really just a reflection of the then-current features that had been developed by Netscape and Microsoft. Fortunately, after 1997 the evolution of HTML was dominated by W3C, in part because Netscape had surrendered from their browser competition with Microsoft. The browsers produced by the two companies have since drifted ever closer to W3C standards.

The latest version of HTML, 4.01 was approved by W3C in late 1999. The XHTML 1.0 standard was approved in early 2000. XHTML 1.0 is a redefinition of HTML 4.01 using XML.[2] The XHTML 1.1 standard was recommended by W3C in May 2001. This standard, primarily a modularization of XHTML 1.0, drops some of the features of its predecessor, most notably frames. XHTML 2.0 was getting close to release at the time of this writing.[3]

The latest versions of the most popular browsers, Microsoft Internet Explorer 7 (IE7) and Firefox 2 (FX2), come close to supporting all of XHTML 1.1.

The addition of presentation details through style sheets in HTML 4.0 made some features of earlier versions obsolete. These features, as well as some others, have been *deprecated*, meaning that they will be dropped from HTML at some time in the future. Deprecating a feature is a warning to users to stop using the feature because it will not be supported forever. Although even the latest releases of browsers still support the deprecated parts of HTML, we do not include descriptions of most of them in this book. The only exception is frames, which are discussed in Section 2.10.

XHTML 1.0 was defined in three levels, one of which, named Transitional, allowed the inclusion of HTML 4.0's deprecated tags and attributes. XHTML 1.1 eliminated these deprecated features. All of the XHTML documents in this book, except those illustrating frames, conform to the XHTML 1.1 standard.

2.1.2 HTML versus XHTML

There are some commonly heard arguments for using HTML rather than XHTML, especially XHTML 1.1. First, because of its lax syntax rules, HTML is much easier to write, whereas XHTML requires a level of discipline many of us naturally resist. Second, because of the huge number of HTML documents available on the Web, browsers will continue to support it as far as one can see into the future. However, some older browsers have problems with some parts of XHTML.

On the other hand, there are strong reasons why one should use XHTML. One of the most compelling is that quality and consistency in any endeavor, be it electrical wiring, software development, or Web-document development, rely on standards. HTML has few syntactic rules, and HTML processors (for example, browsers) do not enforce the rules it does have. Therefore, HTML authors

2. XML (eXtensible Markup Language) is the topic of Chapter 8, "Introduction to XML."
3. The W3C Web site is `http://www.w3.org`.

have a high degree of freedom to create documents using their own syntactic preferences. Because of this, HTML documents lack consistency, both in low-level syntax and overall structure. Conversely, XHTML has strict syntactic rules that impose a consistent structure on all XHTML documents. Furthermore, the fact that there are a large number of poorly structured HTML documents on the Web is a poor excuse for generating more.

Another significant reason for using XHTML is that when you create an XHTML document, its syntactic correctness can be checked, either by an XML browser or by a validation tool (see Section 2.4). This checking process may find errors that could otherwise go undetected until after the document is posted on a site and requested by a client, possibly with only a specific browser.

The argument that XHTML is difficult to write correctly is obviated by the availability of XTHML editors, which provide a simple and effective approach to creating syntactically correct XHTML documents.[4]

It is also possible to convert legacy HTML documents to XHTML documents using software tools. Tidy, which is available at `http://tidy.sourceforge.net`, is one such tool.

The remainder of this chapter provides an introduction to the most commonly used tags and attributes of XHTML 1.1.

2.2 Basic Syntax

The fundamental syntactic units of HTML are called *tags*. In general, tags are used to specify categories of content. For each category, a browser has default presentation specifications for the specified content. The syntax of a tag is the tag's name surrounded by angle brackets (<>). Tag names must be written in all lowercase letters. Most tags appear in pairs: an opening tag and a closing tag. The name of a closing tag is the name of its corresponding opening tag with a slash attached to the beginning. For example, if the tag's name is `p`, the corresponding closing tag is named `/p`. Whatever appears between a tag and its closing tag is the *content* of the tag. A browser display of an XHTML document shows the content of all of the document's tags; it is the information the document is meant to portray. Not all tags can have content.

The opening tag and its closing tag together specify a container for the content they enclose. The container and its content together are called an *element*. For example, consider the following element:

```
<p> This is extremely simple. </p>
```

The paragraph tag, `<p>`, marks the beginning of the content, and the `</p>` tag marks the end of the content of the paragraph element.

Attributes, which are used to specify alternative meanings of a tag, can appear between an opening tag's name and its right-pointed bracket. They are specified in keyword form, which means that the attribute's name appears, fol-

4. One such editor system is available at `http://www.xstandard.com`.

lowed by an equals sign and the attribute's value. Attribute names, like tag names, are written in lowercase letters. Attribute values must be delimited by double quotes.

Comments in programs increase the readability of those programs. Comments in XHTML have the same purpose. They can appear in XHTML in the following form:

```
<!-- anything except two adjacent dashes -->
```

Browsers ignore XHTML comments—they are for people only. Comments can be spread over as many lines as are needed. For example, you could have the following comment:

```
<!-- PetesHome.html
This document describes the home page of Pete's Pickles
-->
```

Besides comments, several other kinds of text may appear in an XHTML document but be ignored by browsers. Browsers ignore all unrecognized tags. They also ignore line breaks. Line breaks that show up in the displayed content can be specified but only with tags designed for that purpose. The same is true for multiple spaces and tabs.

Programmers find XHTML a bit frustrating. In a program, the statements specify exactly what the computer must do. XHTML tags are treated more like suggestions to the browser. If a reserved word is misspelled in a program, the error is usually detected by the language implementation system, and the program is not executed. However, a misspelled tag name results in the tag being ignored by the browser, with no indication to the browser user that anything has been left out. Browsers are even allowed to ignore tags that they recognize. Furthermore, the browser user can configure his or her browser to react to specific tags in different ways.

2.3 Standard XHTML Document Structure

Every XHTML document must begin with an `xml` declaration element that simply identifies the document as being one based on XML. This element includes an attribute that specifies the version number, which is still 1.0. The `xml` declaration usually includes a second attribute, `encoding`, which specifies the encoding used for the document. In this book, we use the Unicode encoding, `utf-8`. Following is the `xml` declaration element, which should be the first line of every XHTML document:

```
<?xml version = "1.0" encoding = "utf-8"?>
```

Note that this declaration must begin in the first character position of the document file.

Immediately following the `xml` declaration element is an SGML `DOCTYPE` command, which specifies the particular SGML document-type definition

(DTD) with which the document complies, among other things.[5] The following command states that the document in which it is included complies with the XHTML 1.1 standard:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
```

A complete explanation of the `DOCTYPE` command requires more effort, both to write and to read, than is justified at this stage of our introduction to XHTML.

XHTML documents must include the four tags `<html>`, `<head>`, `<title>`, and `<body>`. The `<html>` tag identifies the root element of the document. So, XHTML documents always have an `<html>` tag immediately following the `DOCTYPE` command, and they always end with the closing `html` tag, `</html>`. The `html` element includes an attribute, `xmlns`, that specifies the XHTML namespace, as shown in the following:

```
<html xmlns = "http://www.w3.org/1999/xhtml">
```

Although the `xmlns` attribute's value looks like a URL, it does not specify a document. It is just a name that happens to have the form of a URL. Namespaces are discussed in Chapter 7, "Introduction to XML."

An XHTML document consists of two parts, the *head* and the *body*. The `<head>` element contains the head part of the document, which provides information about the document rather than its content. The body of a document provides the content of the document, which itself includes tags and attributes.

The content of the title element is displayed by the browser at the top of its display window, usually in the browser's window title bar.

Standards prior to XHTML 1.1 allowed a document to have either a body element or a frameset element. The Frameset alternative of the XHTML 1.0 DTD standards applies when the document has a frameset element. The Transitional alternative of the XHTML 1.0 standard allows deprecated HTML features to be used. Both of these will be used in Section 2.10.

2.4 Basic Text Markup

This section describes how the text content of an XHTML document can be formatted with XHTML tags. By *formatting*, we mean layout and some presentation details. For now, we will ignore the other kinds of content that can appear in an XHTML document.

2.4.1 Paragraphs

Text is normally organized into paragraphs in the body of a document. In fact, the XHTML standard does not allow text to be placed directly in a document body. Textual paragraphs appear as the content of a paragraph element, speci-

5. A document-type definition specifies the syntax rules for a particular category of XHTML documents.

fied with the tag <p>. In displaying the content of a paragraph, the browser puts as many words as will fit on the lines in the browser window. The browser supplies a line break at the end of each line. As stated in Section 2.2, line breaks embedded in text are ignored by the browser. For example, the following paragraph might[6] be displayed by a browser, as shown in Figure 2.1.

```
<p>
   Mary had
a
   little lamb, its fleece was white as snow. And
 everywhere that
  Mary went, the lamb
 was sure to go.
</p>
```

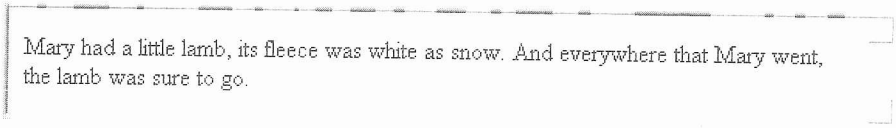


Figure 2.1 Filling lines

Notice that multiple spaces in the source paragraph element are replaced by single spaces in the display of Figure 2.1.

The following is our first example of a complete XHTML document:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- greet.html
     A trivial document
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Our first document </title>
  </head>
  <body>
    <p>
      Greetings from your Webmaster!
    </p>
  </body>
</html>
```

6. We say "might" because the width of the display that the browser uses determines how many words will fit on a line.

Figure 2.2 shows a browser display of `greet.html`.

Figure 2.2 Display of `greet.html`

If the content of a paragraph tag would be displayed at a position other than the beginning of the line, the browser breaks the current line and inserts a blank line. For example, the following line would be displayed, as shown in Figure 2.3.

```
<p> Mary had a little lamb, </p> <p> its fleece was
white as snow. </p>
```

Figure 2.3 The paragraph element

2.4.2 Line Breaks

When the content of a paragraph element would be displayed at a position other than at the beginning of a line, the browser breaks the current line and inserts a blank line. Sometimes text requires a line break without the preceding blank line. This is exactly what the break tag does. The break tag differs from the paragraph tag in that it can have no content and therefore has no closing tag (because it would serve no purpose). The break tag is specified as `<br />`. The slash indicates that the tag is both an opening and closing tag. The space before the slash represents the absent content.[7]

Consider the following:

```
<p>
Mary had a little lamb, <br />
  its fleece was white as snow.
</p>
```

This would be displayed as shown in Figure 2.4.

7. Some older browsers have trouble with the tag `<br/>` but not with `<br />`.

Figure 2.4 Line breaks

2.4.3 Preserving Whitespace

Sometimes it is desirable to preserve the whitespace in text, that is, to prevent the browser from eliminating multiple spaces and ignoring embedded line breaks. This can be specified with the `pre` tag. For example,

```
<p><pre>
Mary
    had a
        little
            lamb
</pre>
```

This would be displayed as shown in Figure 2.5 Notice that the content of the `pre` element is shown in monospace, rather than the default font.

Figure 2.5 The `pre` element

A `pre` element can contain virtually any other tags, except those that cause a paragraph break, such as paragraph elements.

2.4.4 Headings

Text is often separated into sections in documents by beginning each section with a heading. Larger sections sometimes have headings that appear more prominent than headings for sections nested inside them. In XHTML, there are six levels of headings, specified by the tags `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, and `<h6>`, where `<h1>` specifies the highest-level heading. Headings are displayed in a boldface font whose default size depends on the number in the heading tag. On most browsers, `<h1>`, `<h2>`, and `<h3>` use font sizes that are larger than that of the default size of text, `<h4>` uses the default size, and `<h5>` and `<h6>` use smaller sizes. The heading tags always break the current line, so their content always appears on a new line. Browsers usually insert some vertical space before and after all headings.

The following example illustrates the use of headings:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- headings.html
     An example to illustrate headings
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Headings </title>
  </head>
  <body>
    <h1> Aidan's Airplanes (h1) </h1>
    <h2> The best in used airplanes (h2) </h2>
    <h3> "We've got them by the hangarful" (h3) </h3>
    <h4> We're the guys to see for a good used airplane (h4) </h4>
    <h5> We offer great prices on great planes (h5) </h5>
    <h6> No returns, no guarantees, no refunds,
         all sales are final! (h6) </h6>
  </body>
</html>
```

Figure 2.6 shows a browser display of `headings.html`.

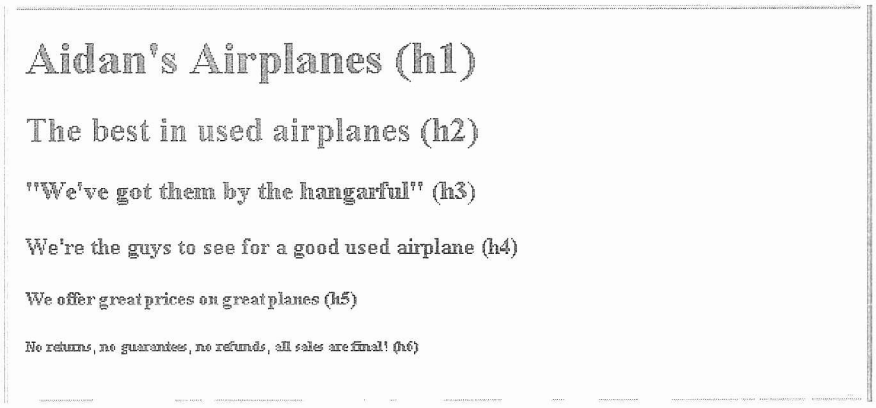


Figure 2.6 Display of `headings.html`

2.4.5 Block Quotations

Sometimes we want a block of text to be set off from the normal flow of text in a document. In many cases, such a block is a long quotation. The `<blockquote>` tag is designed for this situation. Browser designers determine how the content of `<blockquote>` can be made to look different from the surrounding text. In many cases, the block of text is indented, either on the left or right side or both. Another possibility is that the block is set in italic. Consider the following sample document:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- blockquote.html
     An example to illustrate a blockquote
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Blockquotes </title>
  </head>
  <body>
    <p>
      Abraham Lincoln is generally regarded as one of the greatest
      presidents of the U.S. His most famous speech was delivered
      in Gettysburg, Pennsylvania, during the Civil War. This
      speech began with
    </p>
    <blockquote>
      <p>
        "Fourscore and seven years ago our fathers brought forth on
        this continent, a new nation, conceived in Liberty, and
        dedicated to the proposition that all men are created equal.
      </p>
      <p>
        Now we are engaged in a great civil war, testing whether
        that nation or any nation so conceived and so dedicated,
        can long endure."
      </p>
    </blockquote>
    <p>
      Whatever one's opinion of Lincoln, no one can deny the
      enormous and lasting effect he had on the U.S.
    </p>
  </body>
</html>
```

Figure 2.7 shows a browser display of `blockquote.html`.

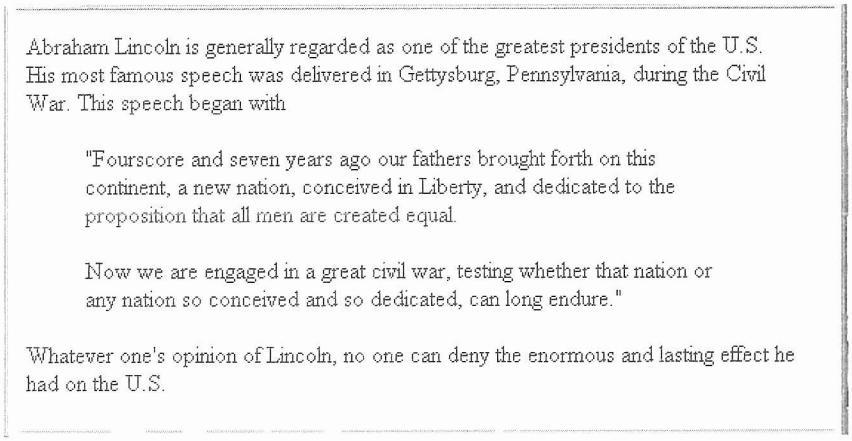


Figure 2.7 Display of `blockquote.html`

2.4.6 Font Styles and Sizes

Early Web designers used a collection of tags to set font styles and sizes. For example, `<i>` specifies italics and `<b>` specifies bold. Since the advent of cascading style sheets (see Chapter 3, "Cascading Style Sheets"), use of these tags has become passe. There are a few tags for fonts that are still in widespread use, called *content-based style tags*. They are called content-based, because the tag indicates the particular kind of text that appears in their content. In the following, three of the most commonly used content-based tags are described.

The emphasis tag, `<em>`, specifies that its textual content is special and should be displayed in some way that indicates this. Most browsers use italics for such content.

The strong tag, `<strong>` is like the emphasis tag, but more so. Browsers often set the content of strong elements in bold.

The `<code>` tag is used to specify a monospace font, usually used for program code. For example,

```
cost = quantity * price
```

would be displayed as shown in Figure 2.8.

Figure 2.8 The `<code>` element

Subscript and superscript characters can be specified by the `<sub>` and `<sup>` tags, respectively. These are not content-based tags. For example,

```
X<sub>2</sub><sup>3</sup> + y<sub>1</sub><sup>2</sup>
```

would be displayed as shown in Figure 2.9.

Figure 2.9 The `<sub>` and `<sup>` elements

Character-modifying tags are not affected by `<blockquote>` except when there is a conflict. For example, if the text content of `<blockquote>` is set in italic and a part of that text is made the content of an `<em>` tag, the `<em>` tag would have no effect.

XHTML tags are categorized as being either block or inline. The content of an *inline* tag appears on the current line. So, an inline tag does not implicitly include a line break. One exception is `br`, which is an inline tag, but its entire purpose is to insert a line break in the content. A *block* tag breaks the current line so that its content appears on a new line. The heading and block quote tags are block tags, whereas `<em>` and `<strong>` are inline tags. In XHTML, block tags cannot appear in the content of inline tags. Therefore, a block tag can never be nested directly in an inline tag. Also, inline tags and text cannot be directly nested in body or form elements. Only block tags can be nested directly in a body or form element. That is why the example, `greet.html`, has the text content of its body nested in a paragraph element.

2.4.7 Character Entities

XHTML provides a collection of special characters that are sometimes needed in a document but cannot be typed as themselves. In some cases, these characters are used in XHTML in some special way, for example `>`, `<`, and `&`. In other cases, the characters do not appear on a keyboard, such as the small raised circle that represents "degrees" in a reference to temperature. Finally, there is the nonbreaking space, which browsers regard as a hard space—they do not squeeze them out like they do other multiple spaces. These special characters are defined as *entities*, which are names for the characters by the browser. An entity in a document is replaced by its associated character by the browser. Table 2.1 lists some of the most commonly used entities.

Table 2.1 Some commonly used entities

Character	Entity	Meaning
&	`&`	Ampersand
<	`<`	Less than
>	`>`	Greater than
"	`"`	Double quote
'	`'`	Single quote (apostrophe)
¼	`¼`	One quarter
½	`½`	One half
¾	`¾`	Three quarters
°	`°`	Degree
(space)	` `	Nonbreaking space

2.4.8 Horizontal Rules

The parts of a document can be separated from each other, making the document easier to read, by placing horizontal lines between them. Such lines are called *horizontal rules*, and the block tag that creates them is `<hr />`. The `<hr />` tag causes a line break (ending the current line) and draws a line across the screen. The browser chooses the thickness, length, and horizontal placement of the line. Typically, browsers display lines that are three pixels thick.

Note again the slash in the `<hr />` tag, indicating that this tag has no content and no closing tag.

2.4.9 The meta Element

The `meta` element is used to provide additional information about a document. It has no content; rather, all of the provided information is specified through attributes. The two attributes that are used to provide information are `name` and `content`. The user makes up a name as the value of the `name` attribute and specifies information through the `content` attribute. One commonly chosen name is `keywords`; the value of the `content` attribute associated with the keywords are those that a document author believes characterizes his or her document. For example:

```
<meta name = "keywords"  content = "binary trees,
linked lists, stacks" />
```

Web search engines use the information provided with the `meta` element to categorize Web documents in their indices. So, if the author of a document

seeks widespread exposure for the document, one or more `meta` elements are included to ensure that it will be found by at least some Web searches. For example, if an entire book were published as a Web document, it might have the following `meta` elements:

```
<meta name = "Title" content = "Don Quixote" />
<meta name = "Author"  content = "Miguel Cervantes" />
<meta  name = "keywords"  content = "novel,
 Spanish literature, groundbreaking work" />
```

2.5 Images

The inclusion of images in a document can dramatically enhance its appearance (although images slow the document-download process considerably for clients who do not have high-speed Internet access). The image is stored in a file, which is specified by an XHTML request. The image is inserted into the display of the document by the browser.

2.5.1 Image Formats

The two most common methods of representing images are Graphic Interchange Format (GIF, pronounced like the first syllable of *jif-fy*) and Joint Photographic Experts Group (JPEG, pronounced *jay-peg*) format. Most contemporary browsers can render images in either of these two formats. Files in both of these formats are compressed to reduce storage needs and provide faster transfer over the Internet.

The GIF format was developed by the CompuServe network service provider for the specific purpose of moving images. It uses 8-bit color representations for pixels, allowing a pixel to have 256 different colors. If you are not familiar with color representations, this may seem to be entirely adequate. However, with the color displays on most contemporary computers, this leaves a huge number of colors that can be displayed but that cannot be represented in a GIF image. Files containing GIF images use the `.gif` (or `.GIF`) extension on their names. GIF images can be made to appear transparent.

The JPEG format uses 24-bit color representations for pixels, which allows JPEG images to include more than 16 million different colors. Files that store JPEG images use the `.jpg` (or `.JPG` or `.jpeg`) extension on their names. The compression algorithm used by JPEG is better at shrinking an image than the one used by GIF. This compression process actually loses some of the color accuracy of the image, but because there is so much to begin with, the loss is rarely discernable by the user. Because of this powerful compression process, even though a JPEG image has much more color information than a GIF image of the same subject, the JPEG image can be smaller than the GIF image. Because of this, JPEG images are often preferred to GIF images. The disadvantage of JPEG is that it does not support transparency.

A third image format is now gaining popularity—Portable Network Graphics (PNG, pronounced *ping*). PNG was designed in 1996 as a free replacement for GIF after the patent owner for GIF, Unisys, suggested it may begin charging royalties for documents that included GIF images.[8] Actually, PNG provides a good replacement for both GIF and JPEG because it has the best characteritics of both (the possibility of transparency, as provided by GIF, and a much larger number of colors than GIF, as with JPEG). One drawback of PNG is that because its compression algorithm does not sacrifice picture clarity, its images require more space than comparable JPEG images.[9] Support for PNG in the earlier IE browsers was unacceptably poor, which kept many developers from using PNG. However, IE7 is much better than IE6 at displaying PNG images (although its support is still not completely correct). Information on PNG can be found at `www.w3.org/Graphics/PNG`.

2.5.2 The <img /> Tag

The image tag, `<img />`, which is an inline tag, specifies an image that is to appear in a document. In its simplest form, the image tag includes two attributes: `src`, which specifies the file containing the image; and `alt`, which specifies text to be displayed when it is not possible to display the image. If the file is in the same directory as the XHTML file of the document, the value of `src` is just the image's filename. In many cases, image files are stored in a subdirectory of the directory where the XHTML files are stored. For example, the image files might be stored in a subdirectory named `images`. If the image file's name is `stars.jpg` and it is stored in the `images` subdirectory, the value of `src` would be as follows:

```
"images/stars.jpg"
```

Some seriously aged browsers are not capable of displaying images. When such a browser finds an `<img />` tag, it simply ignores its content, possibly leaving the user confused by the text in the neighborhood of where the image was supposed to be. Also, graphical browsers, which *are* capable of displaying images, may have image downloading disabled by the browser user. This is done when the Internet connection is slow and the user chooses not to wait for images to be downloaded. It is also done by visually impaired users. In any case, it is helpful to have some text displayed in place of the ignored image. For these reasons, the `alt` attribute is required by XHTML.

Two optional attributes of `img`, `width` and `height`, can be included to specify (in pixels) the size of the rectangle for the image. These can be used to scale the size of the image (that is, to make it larger or smaller). Care must be taken to ensure that the image is not distorted in the resizing. For example, if the image is square, the `width` and `height` attribute values must be equal.

---

8. The patent expired in the United States in 2003.

9. Space is not the direct issue; download time, which depends on file size, is the real issue.

The following is an example of an image element:

```
<img src = "c210.jpg"  alt = "Picture of a Cessna 210" />
```

The following example extends the airplane ad document to include information about a specific airplane and an image of it.

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- image.html
     An example to illustrate an image
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Images </title>
  </head>
  <body>
    <h1> Aidan's Airplanes </h1>
    <h2> The best in used airplanes </h2>
    <h3> "We've got them by the hangarful" </h3>
    <h2> Special of the month </h2>
    <p>
      1960 Cessna 210 <br />
      577 hours since major engine overhaul<br />
      1022 hours since prop overhaul <br /><br />
      <img src = "c210new.jpg"  alt = "Picture of a Cessna 210" />
      <br />
      Buy this fine airplane today at a remarkably low price
      <br />
      Call 999-555-1111 today!
    </p>
  </body>
</html>
```

Figure 2.10 shows a browser display of `image.html`.

There is much more to the `<img />` tag than we have led you to believe. In fact, the `<img />` tag can include up to 30 different attributes. For descriptions of the rest, visit `http://www.w3.org/TR/html401/index/attributes.html`.

2.5.3 XHTML Document Validation

The W3C provides a convenient Web-based way to validate XHTML documents against its standards. The URL of the service is `http://validator.w3.org/file-upload.html`. Figure 2.11 shows a browser display of `file-upload.html`.

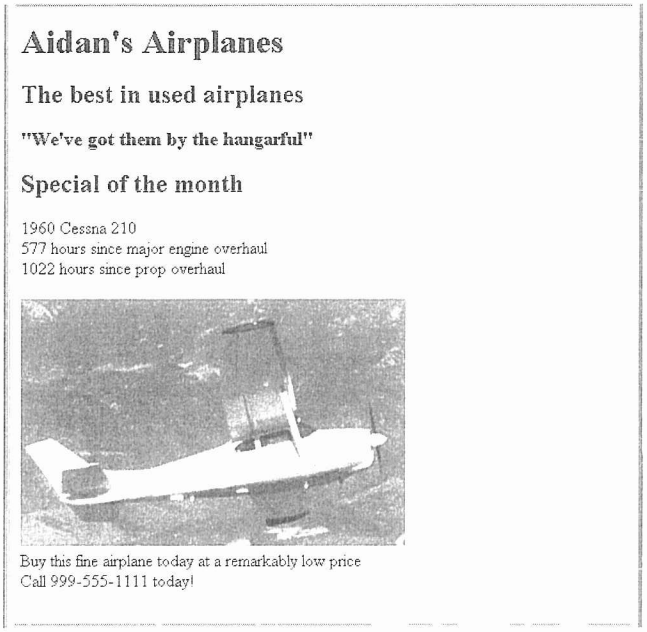


Figure 2.10 Display of `image.html`

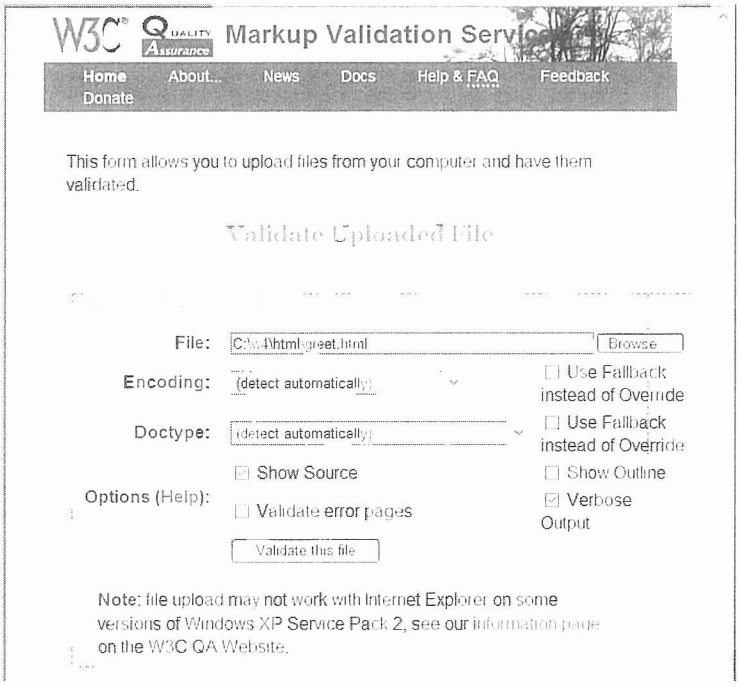


Figure 2.11 Display of `file-upload.html`, the W3C HTML validation

The filename of the document to be validated is entered (including the pathname) or found by browsing. When the `Validate this file` button is pressed, the specified file is uploaded to the `validator` server, where the validation system is run on it. We recommend that the `Show Source` checkbox be checked because that causes the validation system to furnish a listing of the document in which the lines are numbered. These numbers are referenced in the report provided by the validation system.

Figure 2.12 shows a browser display of the document returned by the validation system for our sample document `image.html`. Notice that we cut the source listing off in the figure, simply to prevent the figure from spanning more than one page.

One of the most common errors made in crafting XHTML documents is putting text or elements where they do not belong. For example, putting text directly in a body element is illegal. The XHTML validation system is a valuable tool for producing documents that adhere to W3C standards. The specific standard against which the document is checked is given in the `DOCTYPE` command. Because the `DOCTYPE` command in `image.html` specifies the `xhtml11.dtd` DTD, this document is checked against the XHTML 1.1 standard.

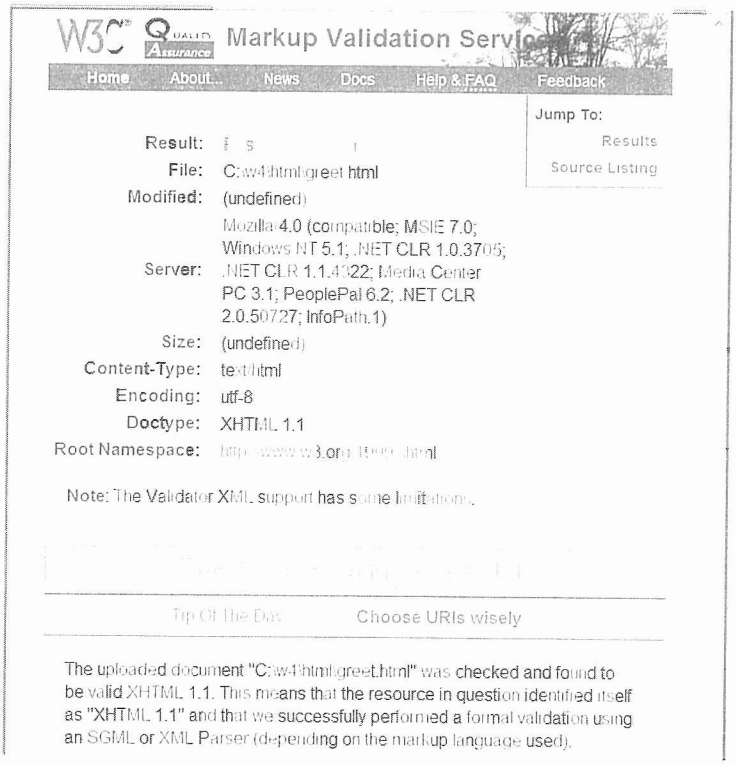


Figure 2.12 HTML validation output for `image.html`

2.6 Hypertext Links

A hypertext link in an XHTML document, which we simply call a *link* here, acts as a pointer to some resource. That resource can be an XHTML document anywhere on the Web, or it may just be another place in the document currently being displayed. It could also be a specific place (rather than the top) in some other document. Without links, Web documents would be boring and tedious to read. There would be no convenient way for the browser user to get from one document to any logically related document. Most Web sites consist of many different documents, all logically linked together. Therefore, links are essential to building any interesting Web site.

2.6.1 Links

A link that points to a different document specifies the address of that document. Such an address might be a filename, a directory path and a filename, or a complete URL. If a link points to a specific place in any document other than the beginning, that place somehow must be marked. Specifying such places is discussed in Section 2.6.2.

Links are specified in an attribute of an anchor tag (`<a>`), which is an inline tag. The anchor tag that specifies a link is called the *source* of that link. The document whose address is specified in a link is called the *target* of that link.

As is the case with many tags, the anchor tag can include many different attributes. However, for creating links only one is required, `href` (an acronym for hypertext `ref`erence). The value assigned to `href` specifies the target of the link. If the target is in another document in the same directory, the target is just the document's filename. If the target document is in some other directory, the UNIX pathname conventions are used. So, an XHTML file named `c210data.html` in a subdirectory of the directory in which the source XHTML file—say, named `airplanes`—is specified in the `href` attribute as `airplanes/c210data.html`. This is the relative method of document addressing. Absolute file addresses could be used in which the entire pathname for the file is given. However, relative links are easier to maintain, especially if a hierarchy of XHTML files must be moved. If the document is on some other machine (not on the server providing the document that includes the link), the complete URL obviously must be used.

The content of an anchor tag, which becomes the clickable link the user sees, is restricted to text, line breaks, images, and headings. Although some browsers allow other nested tags, that is not standard XHTML and should not be used if you want your code to be correctly displayed by all browsers. Links are usually implicitly rendered in a different color than the surrounding text. Sometimes they are also underlined. When the mouse cursor is placed over the anchor-tag content and the left mouse button is pressed, the link is taken by the browser. If the target is a different document, that document is loaded and displayed, replacing the currently displayed document. If the target is in the current document, the document is scrolled by the browser to display the target of

the link. As an example, consider the following document, which adds a link to the document displayed in Figure 2.10:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- link.html
     An example to illustrate a link
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> A link </title>
  </head>
  <body>
    <h1> Aidan's Airplanes </h1>
    <h2> The best in used airplanes </h2>
    <h3> "We've got them by the hangarful" </h3>
    <h2> Special of the month </h2>
    <p>
      1960 Cessna 210 <br />
      <a href = "C210data.html"> Information on the Cessna 210 </a>
    </p>
  </body>
</html>
```

In this case, the target is a complete document that is stored in the same directory as the XHTML document. Figure 2.13 shows a browser display of `link.html`. When the link shown in Figure 2.13 is clicked, the browser displays the screen shown in Figure 2.14.

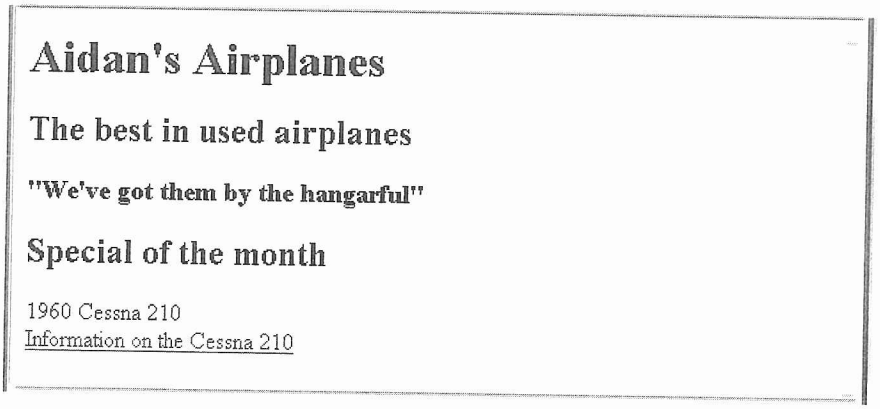


Figure 2.13 Display of `link.html`

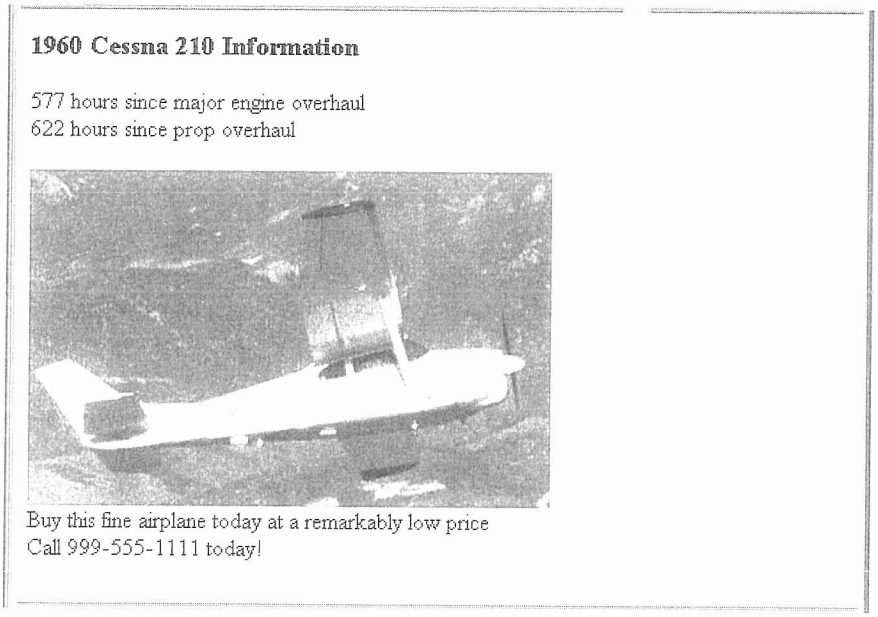


Figure 2.14 Following the link from `link.html`

Links can include images in their content, in which case the browser displays the image with the link:

```
<a href = "c210data.html" >
  <img src = "small-airplane.jpg"
       alt = "An image of a small airplane" />
    Information on the Cessna 210
</a>
```

An image itself can be an effective link (the content of the anchor element). For example, an image of a small house can be used for the link back to the home document of a site. The content of an anchor element for such a link is just the image element.

2.6.2 Targets within Documents

If the target of a link is not at the beginning of a document, it must be some element within a document, in which case there must be some means of specifying it. The target element can include an `id` attribute, which can then be used to identify it in an `href` attribute. Consider the following example:

```
<h2 id = "avionics"> Avionics </h2>
```

Nearly all elements can include an `id` attribute. The value of an `id` attribute must be unique within the document.

If the target is in the same document as the link, the target is specified in the `href` attribute value by preceding the `id` value with a pound sign (#), as in the following example:

```
<a href = "#avionics"> What about avionics? </a>
```

When the `What about avionics?` link is taken, the browser moves the display so that the `h2` element whose `id` is `avionics` is at the top.

When the target is a part or fragment of another document, the name of the part is specified at the end of the URL, separated by a pound sign (#), as in this example:

```
<a href = "AIDAN1.html#avionics"> Avionics </a>
```

2.6.3 Using Links

One common use of links to parts of the same document is to provide a table of contents in which each entry has a link. This provides a convenient way for the user to get to the various parts of the document simply and quickly. Such a table of contents is implemented as a stylized list of links, using the list specification capabilities of XHTML, which are discussed in Section 2.7.

Links exemplify the true spirit of hypertext. The reader can click on links to learn more about a particular subtopic of interest and then return to the location of the link. Designing links requires some care because they can be annoying if the designer tries too hard to convince the user to take them. For example, making them stand out too much from the surrounding text can be distracting. A link should blend into the surrounding text as much as possible so that reading the document without clicking any of the links is easy and natural.

2.7 Lists

We frequently make and use lists in daily life—for example, to-do lists and grocery lists. Likewise, both printed and displayed information is littered with lists. XHTML provides simple and effective ways to specify lists in documents. The primary supported list types are those with which most people are already familiar: unordered lists such as grocery lists and ordered lists such as the assembly instructions for a new bicycle. Definition lists can also be defined. The tags to specify unordered, ordered, and definition lists are described in this section.

2.7.1 Unordered Lists

The `<ul>` tag, which is a block tag, creates an unordered list. Each item in a list is specified with an `<li>` tag (li is an acronym for *l*ist *i*tem). Any tags can appear

in a list item, including nested lists. When displayed, each list item is implicitly preceded with a bullet. For example, consider the following:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- unordered.html
     An example to illustrate an unordered list
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Unordered list </title>
  </head>
  <body>
    <h3> Some Common Single-Engine Aircraft </h3>
    <ul>
      <li> Cessna Skyhawk </li>
      <li> Beechcraft Bonanza </li>
      <li> Piper Cherokee </li>
    </ul>
  </body>
</html>
```

Figure 2.15 shows a browser display of `unordered.html`.

Figure 2.15 Display of `unordered.html`

2.7.2 Ordered Lists

Ordered lists are those in which the order of items is important. This orderedness of a list is shown in the display of the list by the implicit attachment of a sequential value to the beginning of each item. The default sequential values are Arabic numerals, beginning with 1.

An ordered list is created within the block tag `<ol>`. The items are specified and displayed just like those for unordered lists, except that the items in an

ordered list are preceded by sequential values instead of bullets. Consider the following example of an ordered list:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- ordered.html
     An example to illustrate an ordered list
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Ordered list </title>
  </head>
  <body>
    <h3> Cessna 210 Engine Starting Instructions </h3>
    <ol>
      <li> Set mixture to rich </li>
      <li> Set propeller to high RPM </li>
      <li> Set ignition switch to "BOTH" </li>
      <li> Set auxiliary fuel pump switch to "LOW PRIME" </li>
      <li> When fuel pressure reaches 2 to 2.5 PSI, push
           starter button
      </li>
    </ol>
  </body>
</html>
```

Figure 2.16 shows a browser display of `ordered.html`.

Cessna 210 Engine Starting Instructions

1. Set mixture to rich
2. Set propeller to high RPM
3. Set ignition switch to "BOTH"
4. Set auxiliary fuel pump switch to "LOW PRIME"
5. When fuel pressure reaches 2 to 2.5 PSI, push starter button

Figure 2.16 Display of `ordered.html`

As noted earlier, lists can be nested. However, a list cannot be directly nested; that is, an `<ol>` tag cannot immediately follow an `<ol>` tag. Rather, the

nested list must be the content of an <li> element. The following example illustrates nested ordered lists:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- nested_lists.html
     An example to illustrate nested lists
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Nested lists </title>
  </head>
  <body>
    <h3> Aircraft Types </h3>
    <ol>
      <li> General Aviation (piston-driven engines)
        <ol>
          <li> Single-Engine Aircraft
            <ol>
              <li> Tail wheel </li>
              <li> Tricycle </li>
            </ol> <br />
          </li>
          <li> Dual-Engine Aircraft
            <ol>
              <li> Wing-mounted engines </li>
              <li> Push-pull fuselage-mounted engines </li>
            </ol>
          </li>
        </ol> <br />
      </li>
      <li> Commercial Aviation (jet engines)
        <ol>
          <li> Dual-Engine
            <ol>
              <li> Wing-mounted engines </li>
              <li> Fuselage-mounted engines </li>
            </ol> <br />
          </li>
          <li> Tri-Engine
            <ol>
              <li> Third engine in vertical stabilizer </li>
              <li> Third engine in fuselage </li>
            </ol>
```

```
        </li>
      </ol>
    </li>
  </ol>
 </body>
</html>
```

Figure 2.17 shows a browser display of `nested_lists.html`.

Aircraft Types

1. General Aviation (piston-driven engines)
    1. Single-Engine Aircraft
        1. Tail wheel
        2. Tricycle
    2. Dual-Engine Aircraft
        1. Wing-mounted engines
        2. Push-pull fuselage-mounted engines
2. Commercial Aviation (jet engines)
    1. Dual-Engine
        1. Wing-mounted engines
        2. Fuselage-mounted engines
    2. Tri-Engine
        1. Third engine in vertical stabilizer
        2. Third engine in fuselage

Figure 2.17 Display of `nested_lists.html`

One problem with the nested lists shown in Figure 2.17 is that all three levels use the same sequence values. Chapter 3 describes how style sheets can be used to specify different kinds of sequence values for different lists.

The `nested_lists.html` example uses nested ordered lists. There are no restrictions on list nesting, provided the nesting is not direct. For example, ordered lists can be nested in unordered lists and vice versa.

2.7.3 Definition Lists

As the name implies, definition lists are used to specify lists of terms and their definitions, such as in glossaries. A definition list is given as the content of a `<dl>` tag, which is a block tag. Each term to be defined in the definition list is given as the content of a `<dt>` tag. The definitions themselves are specified as the content of `<dd>` tags. The defined terms of a definition list

are usually displayed on the left margin; the definitions are usually shown on the line or lines following the term, which are indented. Consider the following example:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- definition.html
     An example to illustrate definition lists
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Definition lists </title>
  </head>
  <body>
    <h3> Single-Engine Cessna Airplanes </h3>
    <dl>
      <dt> 152 </dt>
      <dd> Two-place trainer </dd>
      <dt> 172 </dt>
      <dd> Smaller four-place airplane </dd>
      <dt> 182 </dt>
      <dd> Larger four-place airplane </dd>
      <dt> 210 </dt>
      <dd> Six-place airplane - high performance </dd>
    </dl>
  </body>
</html>
```

Figure 2.18 shows a browser display of `definition.html`.

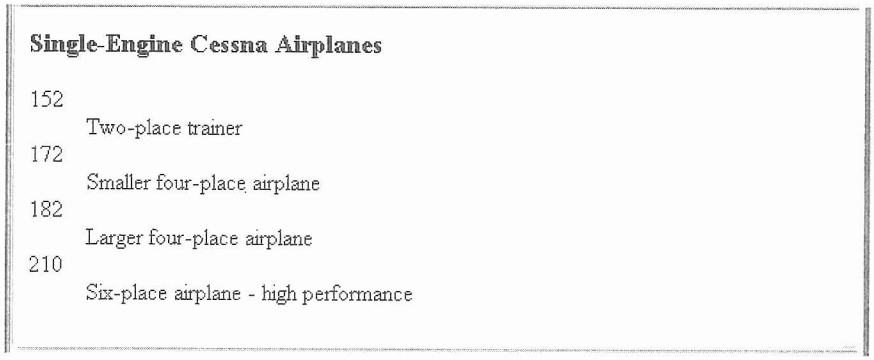


Figure 2.18 Display of `definition.html`

2.8 Tables

Tables are common fixtures in printed documents, books, and of course, Web documents. Tables provide a highly readable way of presenting many kinds of information.

A table is a matrix of rows and columns, in which each intersection of a row and a column is called a *cell*. The cells in the top row often contain column labels; those in the leftmost column often contain row labels; most of the rest of the cells contain the data of the table. The content of a cell can be almost any document element, including text, headings, horizontal rules, images, and nested tables.

2.8.1 Basic Table Tags

A table is specified as the content of the block tag `<table>`. The most common attribute for the `<table>` tag is `border`. There are two kinds of lines in tables, the line around the outside of the whole table is called the *border*, and the lines that separate the cells from each other are called *rules*. A table that does not include the `border` attribute will be a matrix of cells with neither a border nor rules. The browser has default widths for table borders and rules, which is used if the `border` attribute is assigned the value `"border."` Otherwise, a number can be given as `border`'s value, which specifies the border width in pixels. For example, `border = "3"` specifies a border 3 pixels wide. A `border` value of `"0"` specifies no border and no rules. The rule lines are set at 1 pixel when any non-zero `border` value is specified. All table borders are beveled to give a three-dimensional appearance, although this is ineffective when narrow border widths are used.

In most cases, a displayed table is preceded by a title, which is given as the content of a `<caption>` tag, which can immediately follow the opening `<table>` tag. The cells of a table are specified one row at a time. Each row of a table is specified with a row tag, `<tr>`. Within each row, the row label is specified by the table heading tag, `<th>`. Although the `<th>` tag has *heading* in its name, we call these things *labels* to avoid confusion with headings created with the `<hx>` tags. Each data cell of a row is specified with the table data tag, `<td>`. The first row of a table usually has the table's column labels. For example, if a table has three data columns and their column labels are `Apple`, `Orange`, and `Screwdriver`, the first row can be specified by the following:

```
<tr>
  <th> Apple </th>
  <th> Orange </th>
  <th> Screwdriver </th>
</tr>
```

Each data row of a table is specified with a heading tag and one data tag for each data column. For example, the first data row for our work-in-progress table might be as follows:

```
<tr>
  <th> Breakfast </th>
  <td> 0 </td>
  <td> 1 </td>
  <td> 0 </td>
</tr>
```

In tables that have both row and column labels, the upper-left corner cell is often empty. This empty cell is specified with a table header tag that includes no content (either `<th></th>` or just `<th />`).

The following document describes the whole table:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- table.html
     An example of a simple table
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> A simple table </title>
  </head>
  <body>
    <table border = "border">
      <caption> Fruit Juice Drinks </caption>
      <tr>
        <th> </th>
        <th> Apple </th>
        <th> Orange </th>
        <th> Screwdriver </th>
      </tr>
      <tr>
        <th> Breakfast </th>
        <td> 0 </td>
        <td> 1 </td>
        <td> 0 </td>
      </tr>
      <tr>
        <th> Lunch </th>
        <td> 1 </td>
        <td> 0 </td>
        <td> 0 </td>
      </tr>
      <tr>
        <th> Dinner </th>
```

```
        <td> 0 </td>
        <td> 0 </td>
        <td> 1 </td>
      </tr>
    </table>
  </body>
</html>
```

Figure 2.19 shows a browser display of this table.

Fruit Juice Drinks

	Apple	Orange	Screwdriver
Breakfast	0	1	0
Lunch	1	0	0
Dinner	0	0	1

Figure 2.19 Display of `table.html`

2.8.2 The rowspan and colspan Attributes

In many cases, tables have multiple levels of row or column labels in which one label covers two or more secondary labels. For example, consider the display of a partial table shown in Figure 2.20. In this table, the upper-level label "Fruit Juice Drinks" spans the three lower-level label cells. Multiple-level labels can be specified with the `rowspan` and `colspan` attributes.

Fruit Juice Drinks		
Orange	Apple	Screwdriver

Figure 2.20 Two levels of column labels

The `colspan` attribute specification in a table header or table data tag tells the browser to make the cell as wide as the specified number of rows below it in the table. For the previous example, the following code could be used:

```
<tr>
  <th colspan = "3"> Fruit Juice Drinks </th>
</tr>
```

```
<tr>
  <th> Orange </th>
  <th> Apple </th>
  <th> Screwdriver </th>
</tr>
```

If there are fewer cells in the rows above or below the spanning cell than the `colspan` attribute specifies, the browser stretches the spanning cell over the number of cells that populate the column in the table.[10] The `rowspan` attribute of the table heading and table data tags does for rows what `colspan` does for columns.

A table that has two levels of column labels and also has row labels must have an empty upper-left corner cell that spans both the multiple rows of column labels and the multiple columns. Such a cell is specified by including both `rowspan` and `colspan` attributes. Consider the following table specification, which is a minor modification of the previous table:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- cell_span.html
     An example to illustrate rowspan and colspan
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Rowspan and colspan </title>
  </head>
  <body>
    <table border = "border">
      <caption> Fruit Juice Drinks and Meals </caption>
      <tr>
        <td rowspan = "2"> </td>
        <th colspan = "3"> Fruit Juice Drinks </th>
      </tr>
      <tr>
        <th> Apple </th>
        <th> Orange </th>
        <th> Screwdriver </th>
      </tr>
      <tr>
        <th> Breakfast </th>
        <td> 0 </td>
        <td> 1 </td>
```

10. Some browsers add empty row cells to allow the specified span to occur.

```
      <td> 0 </td>
    </tr>
    <tr>
      <th> Lunch </th>
      <td> 1 </td>
      <td> 0 </td>
      <td> 0 </td>
    </tr>
    <tr>
      <th> Dinner </th>
      <td> 0 </td>
      <td> 0 </td>
      <td> 1 </td>
    </tr>
  </table>
 </body>
</html>
```

Figure 2.21 shows a browser display of `cell_span.html`.

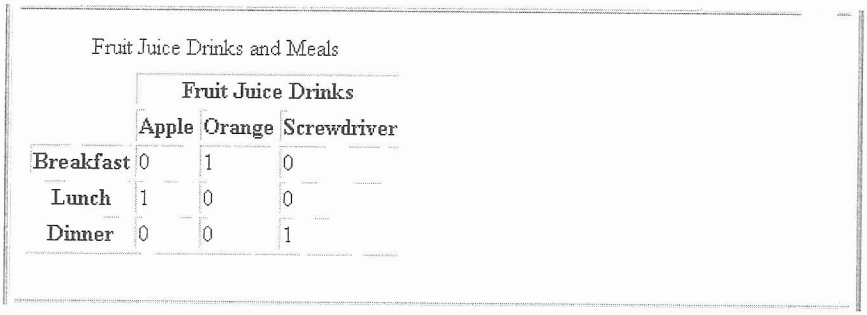


Figure 2.21 Display of `cell_span.html`: multiple-labeled columns and labeled rows

2.8.3 The `align` and `valign` Attributes

The placement of the content within a table cell can be specified with the `align` and `valign` attributes in the `<tr>`, `<th>`, and `<td>` tags. The `align` attribute has the possible values `left`, `right`, and `center`, with the obvious meanings for horizontal placement of the content within a cell. The default alignment for `th` cells is `center`; for `td` cells, it is `left`. If `align` is specified in a `<tr>` tag, it applies to all of the cells in the row. If it is included in a `<th>` or `<td>` tag, it only applies to that cell.

The `valign` attribute of the `<th>` and `<td>` tags has the possible values `top` and `bottom`. The default vertical alignment for both headings and data is `center`. Because `valign` applies only to a single cell, there is never any point in specifying `center`.

The following example illustrates the `align` and `valign` attributes:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- cell_align.html
     An example to illustrate align and valign
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Alignment in cells </title>
  </head>
  <body>
    <table border = "border">
      <caption> The align and valign attributes </caption>
      <tr align = "center">
        <th> </th>
        <th> Column Label </th>
        <th> Another One </th>
        <th> Still Another One </th>
      </tr>
      <tr>
        <th> align </th>
        <td align = "left"> Left </td>
        <td align = "center"> Center </td>
        <td align = "right"> Right </td>
      </tr>
      <tr>
        <th> <br /> valign <br /> <br /> </th>
        <td> Default </td>
        <td valign = "top"> Top </td>
        <td valign = "bottom"> Bottom </td>
      </tr>
    </table>
  </body>
</html>
```

Figure 2.22 shows a browser display of `cell_align.html`.

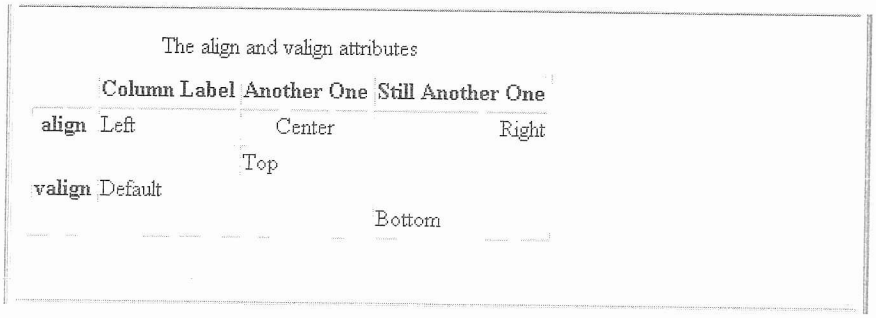


Figure 2.22 Display of `cell_align.html`: the `align` and `valign` attributes

2.8.4 The cellpadding and cellspacing Attributes

The table tag has two attributes that can be used to specify the spacing between the content of a table cell and the cell's edge and the spacing between adjacent cells. The `cellpadding` attribute is used to specify the spacing between the content of a cell and the inner walls of the cell. This is often used to prevent text in a cell from being too close to the edge of the cell. The `cellspacing` attribute is used to specify the distance between cells in a table.

The following document, `space_pad.html`, illustrates the `cellpadding` and `cellspacing` attributes:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- space_pad.html
     An example that illustrates the cellspacing and
     cellpadding table attributes
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Cell spacing and cell padding </title>
  </head>
  <body>
    <b>Table 1 (space = 10, pad = 30) </b><br /><br />
    <table border = "5"  cellspacing = "10"  cellpadding = "30">
      <tr>
        <td> Small spacing, </td>
        <td> large padding </td>
      </tr>
    </table>
```

```
    <br /><br /><br /><br />
    <b>Table 2 (space = 30, pad = 10) </b><br /><br />
    <table border = "5"  cellspacing = "30"  cellpadding = "10">
      <tr>
        <td> Large spacing, </td>
        <td> small padding </td>
      </tr>
    </table>
  </body>
</html>
```

Figure 2.23 shows a browser display of `space_pad.html`.

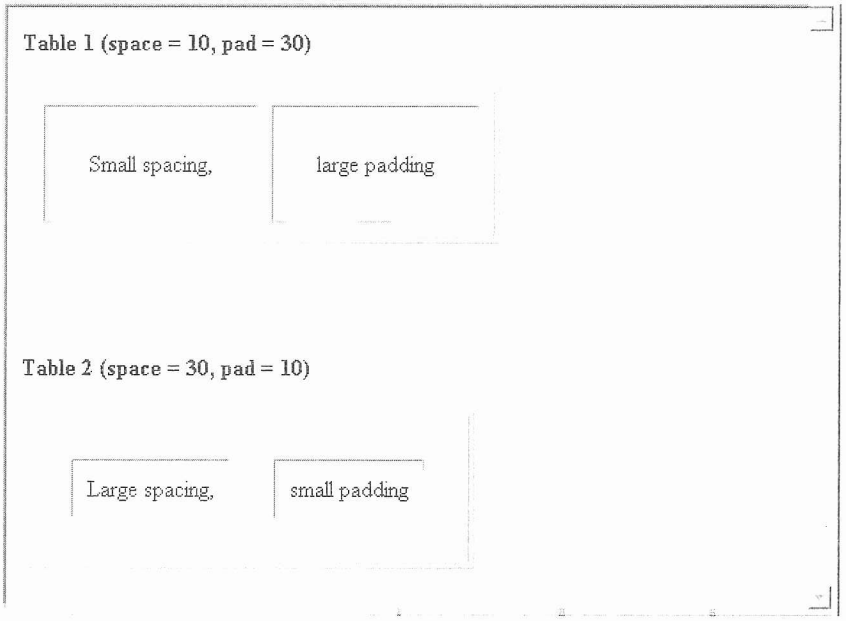


Figure 2.23 Display of `space_pad.html`

2.8.5 Table Sections

Tables naturally occur in two and sometimes three parts: header, body, and footer (not all tables have a natural footer). These three parts can be denoted in XHTML with the `thead`, `tbody`, and `tfoot` elements. The header includes the column labels, regardless of number of levels in those labels. The body includes the data of the table, including the row labels. The footer, when it appears, sometimes has the column labels repeated after the body. In some tables, the footer contains totals for the columns of data above. A table can have

multiple body sections, in which case the browser may delimit them with horizontal lines that are thicker than the rule lines within a body section.

2.9 Forms

The most common way for a user to communicate information from a Web browser to the server is through a form. Forms, which are modeled on the paper forms that people continually are required to fill out, can be described in XHTML and displayed by the browser. XHTML provides tags to generate the commonly used objects on a screen form. These objects are called *controls* or *widgets*. There are controls for single-line and multiple-line text collection, checkboxes, radio buttons, and menus, among others. All control tags are inline tags. Most controls are used to gather information from the user in the form of either text or button selections. Each control can have a value, usually given through user input. Together, the values of all of the controls (that have values) in a form are called the *form data*. Every form requires a *Submit* button (see Section 2.9.5). When the user clicks the *Submit* button, the form data is encoded and sent to the Web server for processing. Form processing is discussed in several subsequent chapters (Chapters 9, 10, 11, and 12).

2.9.1 The <form> Tag

All of the components of a form appear in the content of a `<form>` tag. `<form>`, which is a block tag, can have several different attributes, only one of which, `action`, is required. The `action` attribute specifies the URL of the application on the Web server that is to be called when the user clicks the *Submit* button. In this chapter, our examples of form elements will not have corresponding application programs, so the value of their `action` attributes will be the empty string (`""`).

The `method` attribute of `<form>` specifies one of the two techniques, `get` or `post`, used to pass the form data to the server. `get` is the default, so if no `method` attribute is given in the `<form>` tag, `get` will be used. The alternative technique is `post`. In both techniques, the form data is coded into a text string when the user clicks the *Submit* button.

The `get` and `post` techniques are further discussed in Chapter 9, "Using Perl for CGI Programming."

2.9.2 The <input> Tag

Many of the commonly used controls are specified with the inline tag `<input>`, which is used for text, passwords, checkboxes, radio buttons, and the action buttons *Reset*, *Submit*, and *plain*. The text, password, checkboxes, and radio controls are discussed in this section. The action butons are discussed in Section 2.9.5.

The one attribute of `<input>` that is required for all of the controls discussed in this section is `type`, which specifies the particular kind of control. The

control's kind is its type name, such as `checkbox`. All of the previously listed controls except *Reset* and *Submit* also require a `name` attribute, which becomes the name of the value of the control within the form data. The controls for checkboxes and radio buttons require `value` attributes, which initializes the value of the control.

A text control, which we usually refer to as a text box, creates a horizontal box into which the user can type a line of text. Text boxes are often used to gather information from the user, such as the user's name or address. The default size of a text box is often 20 characters. Because the default size can vary among browsers, it is a good idea to include a size on each text box. This is done with the `size` attribute of `<input>`. If the user types more characters than will fit in the box, the box is scrolled. If you do not want the box to be scrolled, you can include the `maxlength` attribute to specify the maximum number of characters that the browser will accept in the box. Any additional characters are ignored. For example, consider the following text box:

```
<form action = "">
  <p>
    <input type = "text"  name = "Name"  size = "25" />
  </p>
</form>
```

Suppose the user typed the following line:

```
Alfred Paul von Frickenburger
```

The text box would collect the whole string, but the string would be scrolled to the right, leaving the following shown in the box:

```
ed Paul von Frickenburger
```

The left end of the line would be part of the value of `Name`, even though it does not appear in the box. The ends of the line can be viewed in the box by moving the cursor off the ends of the box.

Notice that controls cannot appear directly in the form content—they must be placed in some block container, such as a paragraph.

Now consider a similar text box that includes a `maxlength` attribute.

```
<form action = "">
  <p>
    <input type = "text"  name = "Name"  size = "25"
           maxlength = "25" />
  </p>
</form>
```

If the user typed the same name as in the previous example, the resulting value of the `Name` text box would be as follows:

```
Alfred Paul von Frickenbu
```

No matter what was typed after the u in that person's last name, the value of `Name` would be as shown.

If the contents of a text box should not be displayed when it is entered by the user, a password control can be used. For example:

```
<input type = "password"  name = "myPassword"
       size = "10" maxlength = "10" />
```

In this case, regardless of what characters are typed into a password control, only bullets or asterisks are displayed by the browser.

There are no restrictions on the characters that can be typed into a text box. So, the string `"?!34,:"` could be entered into the text box meant for names. Therefore, the entered contents of text boxes nearly always must be validated, either on the browser or on the server to which the form data is passed for processing.

Text boxes, as well as most other control elements, should be labeled. Labeling could be done by simply inserting text into the approximate places in the form. For example:

```
Phone: <input type = "text"  name = "phone" />
```

This effectively labels the text box, but there are several ways the labeling could be better. For one thing, there is no connection between the label and the control. Therefore, they could become separated in maintenance changes to the document. A control and its label can be connected by putting the control and its label in the content of a label element. For example:

```
<label> Phone: <input type = "text"  name = "phone" />
</label>
```

Now, the text box and its label are encapsulated together. There are several other benefits of this approach to labeling controls. First, browsers often render the text content of a label element differently to make it stand out. Second, if the text content of a label element is selected, the cursor is implicitly moved to the control. This is an aid to new Web users. Third, the text content of a label element can be rendered by a speech synthesizer on the client machine when selected. This can be a great aid to a user with a visual disability.

Checkbox and radio controls are used to collect multiple-choice input from the user. A checkbox control is a single button that is either on or off (checked or not). If a checkbox button is on, the value associated with the name of the button is the string assigned to its `value` attribute. A checkbox button does not contribute to the form data if it is off. Every checkbox button requires a `name` attribute and a `value` attribute in its `<input>` tag. For form processing on the server, the name identifies the button and the value is its value (if it is checked). The attribute `checked`, which is assigned the value `checked`, specifies that the checkbox button is initially on. In many cases, checkboxes appear in lists, with every one in the list having the same name. Checkbox elements should appear in

label elements, for the same reasons that text boxes should. Consider the following example:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- checkbox.html
     An example to illustrate a checkbox
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Checkboxes </title>
  </head>
  <body>
    <p>
      Grocery Checklist
    </p>
    <form action = "">
      <p>
        <label> <input type = "checkbox"  name = "groceries"
              value = "milk"  checked = "checked" /> Milk </label>
        <label> <input type = "checkbox"  name = "groceries"
              value = "bread" /> Bread </label>
        <label> <input type = "checkbox"  name = "groceries"
              value= "eggs" /> Eggs </label>
      </p>
    </form>
  </body>
</html>
```

Figure 2.24 shows a browser display of `checkbox.html`.

Figure 2.24 Display of `checkbox.html`

If the user does not turn on any of the checkbox buttons in our example, `milk` will be the value for `groceries` in the form data. If the `milk` checkbox is left on and the `eggs` checkbox is also turned on by the user, the values of `groceries` in the form data would be `milk` and `eggs`.

Radio buttons are closely related to checkbox buttons. The difference between a group of radio buttons and a group of checkboxes is that only one radio button can be on or pressed at any time. Every time a radio button is pressed, the button in the group that was previously on is turned off. Radio buttons are named after the mechanical push buttons on the radios of cars of the 1950s—when you pushed one button on such a radio, the previously pushed button was mechanically forced out. The `type` value for radio buttons is `radio`. All radio buttons in a group must have the `name` attribute set in the `<input>` tag, and all radio buttons in a group have the same name. A radio button definition may specify which button is to be initially in the pressed, or on, state. This is indicated by including the `checked` attribute, set to the value `checked`, in the `<input>` tag of the button's definition. If no radio button in a group is specified as being checked, the browser usually checks the first button in the group. Consider the following radio button example:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- radio.html
     An example to illustrate radio buttons
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Radio </title>
  </head>
  <body>
    <p>
      Age Category
    </p>
    <form action = "">
      <p>
        <label><input type = "radio"  name = "age"
               value = "under20" checked = "checked" />
               0-19 </label>
        <label><input type = "radio"  name = "age"
               value = "20-35" /> 20-35 </label>
        <label><input type = "radio"  name = "age"
               value = "36-50" /> 36-50 </label>
        <label><input type = "radio"  name = "age"
               value = "over50" /> Over 50 </label>
      </p>
    </form>
  </body>
</html>
```

Figure 2.25 shows a browser display of `radio.html`.

Figure 2.25 Display of `radio.html`

2.9.3 The <select> Tag

Checkboxes and radio buttons are effective methods for collecting multiple-choice data from a user. However, if the number of possible choices is large, the displayed form becomes too long to display. In these cases, a menu should be used. A menu is specified with a `<select>` tag (rather than the `<input>` tag). There are two kinds of menus: those in which only one menu item can be selected at a time (which are related to radio buttons) and those in which multiple menu items can be selected at a time (which are related to checkboxes). The default option is the one related to radio buttons. The other option can be specified by adding the `multiple` attribute, which takes the value `"multiple,"` to the `<select>` tag. When only one menu item is selected, the value sent in the form data is the value of the `name` attribute of the `<select>` tag and the chosen menu item. When multiple menu items have been selected, the value for the menu in the form data includes all selected menu items. If no menu item is selected, no value for the menu is included in the form data. The `name` attribute, of course, is required in the `<select>` tag.

The `size` attribute can be included in the `<select>` tag. `size` specifies the number of menu items that are to be displayed for the user. If no `size` attribute is specified, the value `1` is used. If the value for the `size` attribute is `1` and `multiple` is not specified, just one menu item is displayed with a downward scroll arrow. If the scroll arrow is clicked, the menu is displayed as a pop-up menu. If either `multiple` is specified or the `size` attribute is set to a number larger than `1`, the menu is usually displayed as a scrolled list.

Each of the items in a menu is specified with an `<option>` tag, nested in the select element. The content of an `<option>` tag is the value of the menu item, which is just text (no tags may be included). The `<option>` tag can include the `selected` attribute, which specifies that the item is preselected. The value assigned to `selected` is `"selected."` This preselection can be overridden by the user. The following document describes a menu with the default value (`1`) for `size:`

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- menu.html
     An example to illustrate menus
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Menu </title>
  </head>
  <body>
    <p>
      Grocery Menu - milk, bread, eggs, cheese
    </p>
    <form action = "">
      <p>
        With size = 1 (the default)
        <select name = "groceries">
          <option> milk </option>
          <option> bread </option>
          <option> eggs </option>
          <option> cheese </option>
        </select>
      </p>
    </form>
  </body>
</html>
```

Figure 2.26 shows a browser display of `menu.html`. Figure 2.27 shows a browser display of `menu.html` after clicking the scroll arrow. Figure 2.28 shows a browser display of `menu.html` after modification to set `size` to `"2."`

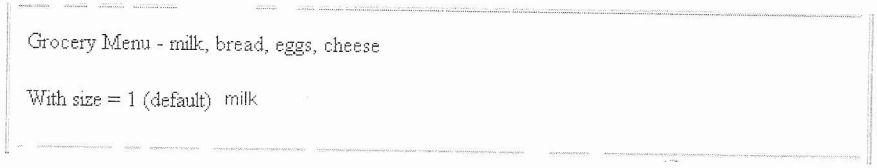


Figure 2.26 Display of `menu.html` (default `size` of 1)

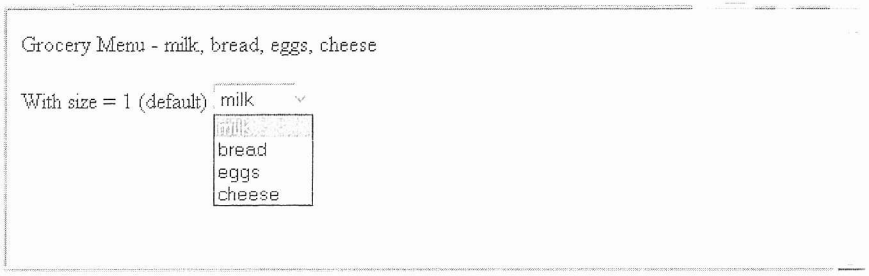


Figure 2.27 Display of `menu.html` after the scroll arrow is clicked

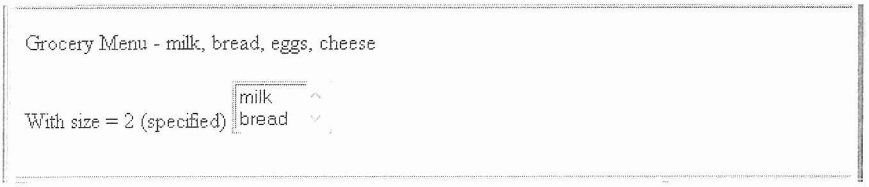


Figure 2.28 Display of `menu.html` with `size` set to 2

When the `multiple` attribute of the `<select>` tag is set, adjacent options can be chosen by dragging the mouse cursor over them while the left mouse button is held down. Nonadjacent options can be selected by clicking them while holding down the keyboard Ctrl key.

2.9.4 The <textarea> Tag

In some situations, a multiline text area is needed. The `<textarea>` tag is used to create such a control. The text typed into the area created by `<textarea>` is not limited in length, and there is implicit scrolling when needed, both vertically and horizontally. The default size of the visible part of the text in a text area is often quite small, so the `rows` and `cols` attributes should usually be included and set to reasonable sizes. If some default text is to be included in the text area, it can be included as the content of the text area element. The following document describes a text area whose window is 40 columns wide and 3 lines tall:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- textarea.html
     An example to illustrate a textarea
     -->
```

```
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Textarea </title>
  </head>
  <body>
    <p>
      Please provide your employment aspirations
    </p>
    <form action = "handler">
      <p>
        <textarea name = "aspirations"  rows = "3"  cols = "40">
          (Be brief and concise)
        </textarea>
      </p>
    </form>
  </body>
</html>
```

Figure 2.29 shows a browser display of `textarea.html` after some text has been typed into the area.

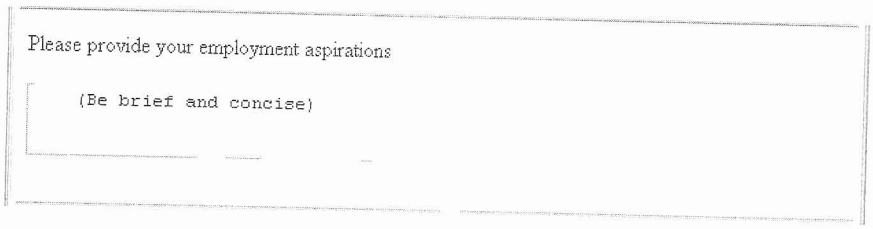


Figure 2.29 Display of `textarea.html` after some text entry

2.9.5 The Action Buttons

The *Reset* button clears all of the controls in the form to their initial states. The *Submit* button has two actions: First, the form data is encoded and sent to the server. Second, the server is requested to execute the server-resident program specified in the `action` attribute of the `<form>` tag. The purpose of such a server-resident program is to process the form data and return some response to the user. Every form requires a *Submit* button. The *Submit* and *Reset* buttons are created with the `<input>` tag, as is illustrated in the following example:

```
<form action = "">
  <p>
    <input type = "submit"  value = "Submit Form" />
    <input type = "reset"  value = "Reset Form" />
  </p>
</form>
```

Figure 2.30 shows a browser display of *Submit* and *Reset* buttons.

Figure 2.30 *Submit* and *Reset* buttons

A *plain* button has the type `button`. *Plain* buttons are used to choose an action.

2.9.6 A Complete Form Example

The following document describes a form for taking sales orders for popcorn. Three text boxes are used at the top of the form to collect the buyer's name and address. These are placed in a borderless table to force the text boxes to align vertically. A second table is used to collect the actual order. Each row of this table names a product with the content of a `<td>` tag, displays the price with another `<td>` tag, and uses a text box with `size` set to `2` to collect the quantity ordered. The payment method is input by the user through one of four radio buttons.

Notice that none of the input controls in this document are embedded in label elements. The reason is that table elements cannot be lebeled, except with the row and column labels

Tables present special problems for the visually impaired. The best solution to this is to use style sheets (see Chapter 3) instead of tables to lay out tabular information. This will be further discussed in Chapter 5.

```
<?xml version = "1.0"  encoding = "utf-8" ?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- popcorn.html
     This describes a popcorn sales form document>
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Popcorn Sales Form </title>
  </head>
  <body>
    <h2> Welcome to Millenium Gymnastics Booster Club Popcorn
         Sales
    </h2>

<!-- The next line gives the address of the CGI program -->
    <form action = "">
```

```
<!-- A borderless table of text widgets for name and address -->
      <table>
        <tr>
          <td> Buyer's Name: </td>
          <td> <input type = "text"  name = "name"
                      size = "30" />
          </td>
        </tr>
        <tr>
          <td> Street Address: </td>
          <td> <input type = "text"  name = "street"
                      size = "30" />
          </td>
        </tr>
        <tr>
          <td> City, State, Zip: </td>
          <td> <input type = "text"  name = "city"
                      size = "30" />
          </td>
        </tr>
      </table>
    <p />

<!-- A bordered table for item orders -->
      <table border = "border">

<!-- First, the column headings -->
        <tr>
          <th> Product Name </th>
          <th> Price </th>
          <th> Quantity </th>
        </tr>

<!-- Now, the table data entries -->
        <tr>
          <td> Unpopped Popcorn (1 lb.) </td>
          <td> $3.00 </td>
          <td> <input type = "text"  name = "unpop"
                      size ="2" />
          </td>
        </tr>
        <tr>
          <td> Caramel Popcorn (2 lb. cannister) </td>
          <td> $3.50 </td>
          <td> <input type = "text"  name = "caramel"
                      size = "2" />
```

```
          </td>
        </tr>
        <tr>
          <td> Caramel Nut Popcorn (2 lb. cannister) </td>
          <td> $4.50 </td>
          <td> <input type = "text"  name = "caramelnut"
                      size = "2" />
          </td>
        </tr>
        <tr>
          <td> Toffey Nut Popcorn (2 lb. cannister) </td>
          <td> $5.00 </td>
          <td> <input type = "text"  name = "toffeynut"
                      size = "2" />
          </td>
        </tr>

      </table>
      <p />

<!-- The radio buttons for the payment method -->
      <h3> Payment Method: </h3>
      <p>
        <label> <input type = "radio"  name = "payment"
                       value = "visa"  checked = "checked" />
                       Visa
        </label>
        <br />
        <label> <input type = "radio"  name = "payment"
                       value = "mc" /> Master Card
        </label>
        <br />
        <label> <input type = "radio"  name = "payment"
                       value = "discover" /> Discover
        </label>
        <br />
        <label> <input type = "radio"  name = "payment"
                       value = "check" /> Check
        </label>
        <br />
      </p>

<!-- The submit and reset buttons -->
      <p>
        <input type = "submit"  value = "Submit Order" />
```

```
        <input type = "reset"  value = "Clear Order Form" />
      </p>
    </form>
  </body>
</html>
```

Figure 2.31 shows a browser display of `popcorn.html`.

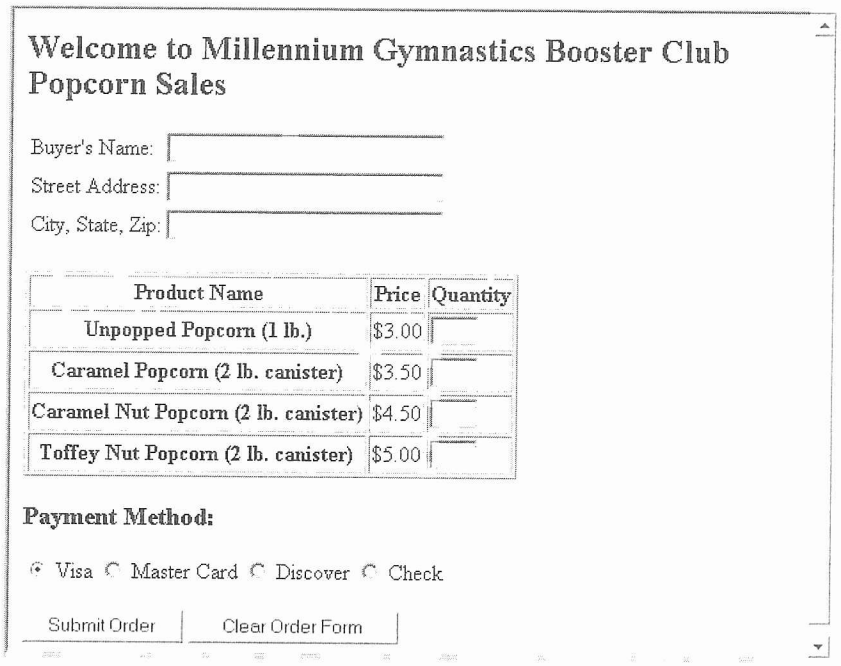


Figure 2.31 Display of `popcorn.html`

In Chapter 9 a Perl program is developed to process the form data from the popcorn form example. Chapter 11, "Introduction to PHP," has a PHP script for processing the data from the same form.

2.10 Frames

The browser display window can be used to display more than one document at a time. The window can be divided into rectangular areas, each of which is a *frame*. Each frame is capable of displaying its own document. Frames can be used for a number of different display situations. Among the most common of these is having a table of contents displayed in one frame and parts of the main document displayed in another. The table of contents can include links that,

when followed, lead to the targeted parts of the main document being displayed in the other frame. Using frames, the table of contents can be displayed while the various documents are being displayed.

There are some problems with frames, which led W3C to begin discouraging their use as early as 1999 (with the release of HTML 4.0). They are entirely left out of XHTML 1.1. This means documents that include frames cannot be validated using the XHTML 1.1 standard. However, frames are commonly found in legacy HTML documents. Furthermore, they are still being included in new documents and are still supported by the latest versions of the popular browsers. Furthermore, work is now in progress at W3C to develop a new form of frames, called XFrames, so the interest in frames lives on. Therefore, we retain our discussion of frames.

2.10.1 Framesets

The number of frames and their layout in the browser window are specified with the `<frameset>` tag. A frameset element takes the place of the body element in a document. A document has either a body or a frameset but cannot have both.

The `<frameset>` tag must have either a `rows` or a `cols` attribute, and often they have both. The `rows` attribute specifies the number of rows of frames that will occupy the window. There are three kinds of values for rows: numbers, percentages, and asterisks. Normally, two or more values, separated by commas, are given in a quoted string. When a number is used as a value, it specifies the height of one row in pixels. A percentage is given as a number followed immediately by a percent sign. When used, a percent value specifies the percentage of the total browser window height that a row should occupy. When an asterisk is used as the value of `rows`, it means the remainder of the window height.

Consider the following example:

```
<frameset rows = "200, 300, 400">
```

This frameset will have three rows of frames. Because no `cols` attribute was included, the frames extend over the entire width of the browser window. If the window height happens to be 900 pixels, the three rows will have the heights 200, 300, and 400 pixels. If the window height is not exactly 900, the actual height will be divided so that the first frame will have $\frac{2}{9}$ of the height, the second $\frac{1}{3}$ of the height, and the last one $\frac{4}{9}$ of the height. Because different displays have different capabilities, it is more practical to specify rows with percentages, as in the following example:

```
<frameset rows = "22%, 33%, 45%">
```

The frameset specified in the following example has the same dimensions as the previous one:

```
<frameset rows = "22%, 33%, *">
```

If two asterisks are given in the `rows` attribute value, they each get half of what remains of the height of the window.

The `cols` attribute is very much like the `rows` attribute, except that it specifies the number of columns of frames. For example, the following tag specifies that the window is to have six frames in three equal-height rows and two columns:

```
<frameset rows = "33%, 33%, 33%"  cols = "25%, *">
```

Figure 2.32 shows a browser display of the window described by this frameset. The documents that appear in the frames in Figure 2.32 contain nothing more than frame labels.

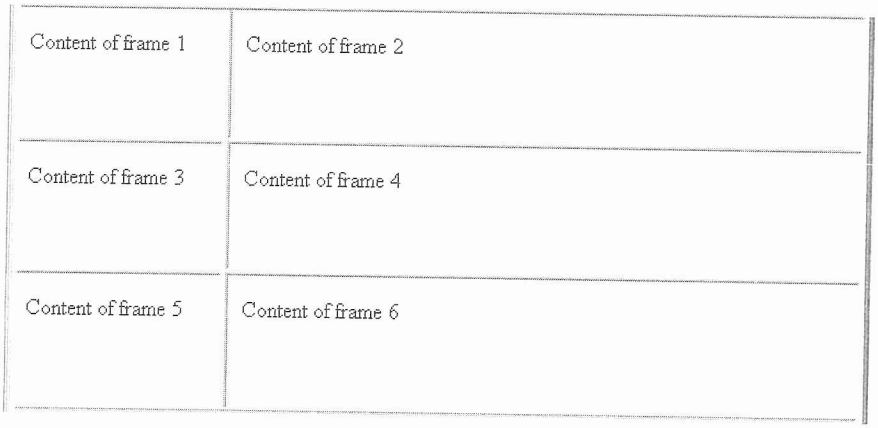


Figure 2.32 A simple frameset of six frames

2.10.2 Frames

The content of a frame is specified with the `<frame>` tag, which can appear only in the content of a frameset element. Each frame defined in a frameset has an associated `<frame>` tag that gives the filename of a document that supplies its content. The sequence of `<frame>` tags in a frameset is important because the order dictates which frame gets which content. The frames in the frameset appear by rows. For example, if the frameset has two rows and two columns, the first two frames fill the two columns of the first row of frames. The content of a frame is specified as the value of the `src` attribute in the `<frame>` tag. For example:

```
<frame src = "apples.html">
```

If a `<frame>` tag has no `src` attribute, the browser displays an empty frame. If the frameset specifies more frames than `<frame>` tags, the unspecified frames are displayed as empty frames. Empty frames are rarely useful because they cannot be filled later.

If the content of a frame does not fit into the given frame, scroll bars are implicitly included. If you want a frame to have scroll bars, regardless of the size of its content, the `<frame>` attribute `scrolling` can be set to `yes`.

If a `<frame>` tag includes a `name` attribute, the content of its associated frame can be changed by the selection of a link in some other frame that specifies that name. For example, consider the following simple document:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//W3C/DTD XHTML 1.0 Frameset//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-frameset.dtd">

<!-- frames.html
     An example to illustrate frames
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Frames </title>
  </head>
  <frameset cols = "20%, *">
    <frame src = "contents.html" />
    <frame src = "fruits.html"  name = "descriptions" />
  </frameset>
</html>
```

This document has two frames. The left frame displays `contents.html`; the right frame displays `fruits.html`. Because the second frame has a name, it can be the target of a link from another frame (in this case, `contents.html`, shown `below`).

Notice that the `frames.html` document uses a different `DOCTYPE` than previous examples. This is because frames are not included in XHTML 1.1. In XHTML 1.0, frames were specified with the Frameset alternative DTD of XHTML 1.0.

The `contents.html` document is a list of links to the fruit description documents. Each link must give both an `href` attribute for the document filename and a `target` attribute to specify the name of the frame in which the document is to be displayed. For example, consider the following document:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<!-- contents.html
     The contents of the first frame of frames.html,
     which is the table of contents for the second frame
     -->
```

```
<html>
<head> <title> Table of Contents Frame </title>
</head>
<body>
<h4> Fruits </h4>
<ul>
    <li> <a href = "apples.html"  target = "descriptions">
            apples </a>
    </li>
    <li> <a href = "bananas.html"  target = "descriptions">
            bananas </a>
    </li>
    <li> <a href = "oranges.html"  target = "descriptions">
            oranges </a>
    </li>
</ul>
</body>
</html>
```

Yet another `DOCTYPE` is used in `contents.html`. This is because its particular use of frames was deprecated in HTML 4.0.[11] The standard used here, XHTML 1.0 Transitional, allows deprecated features of HTML. Recall that frames were completely removed from XHTML 1.1.

Next, consider the `fruits.html` document, which is the initial document displayed in the second frame:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- fruits.html
     The initial contents of the second frame
     of frames.html - a general description of fruit
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> General Information on Fruits </title>
  </head>
  <body>
    <p>
      A fruit is the mature ovary in a flowering plant.
      Fruit is classified by several characteristics, the
      most important being the number of ovaries included.
```

11. Specifically, the `target` attribute of the anchor tag, as used in `contents.html`, was deprecated.

```
      If only a single ovary is included, it is called a
      simple fruit.
    </p>
  </body>
</html>
```

Figure 2.33 shows a browser display of `frames.html`.

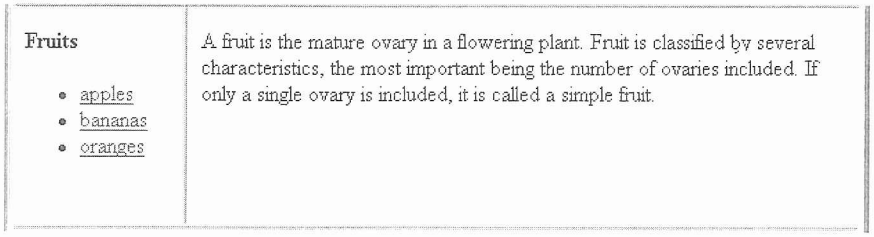


Figure 2.33 Display of `frames.html`

Assuming that `bananas.html` has been defined, after choosing the `bananas` link from the document `contents.html`, the resulting browser display may appear as shown in Figure 2.34.

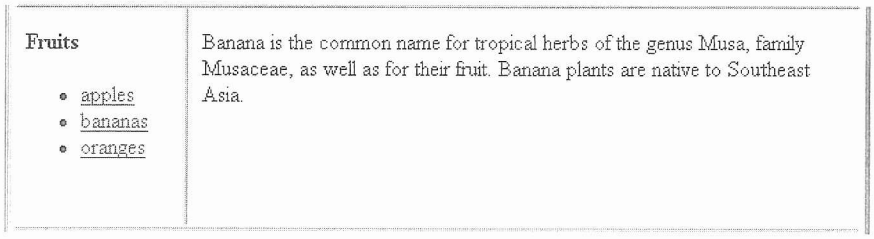


Figure 2.34 Display of `frames.html` after taking the `bananas` link

Frames, as we have described them so far, are actually relatively boring. Although they allow multiple documents to be displayed in different parts of the browser display at the same time, their layout is tediously regular. Fortunately, frames can be arranged in more interesting ways. The mechanism to support this is simple—framesets can be nested. For example, the outermost frameset can define some number of columns. Each of these columns then can be divided with a nested frameset into whatever collection of frames is useful. The following example, `nested_frames.html`, illustrates this approach to dividing the display into irregular frames.

Figure 2.35 shows a browser display of `nested_frames.html`. As was the case with the frames in Figure 2.32, the frames here contain only labels that identify them.

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Frameset//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-frameset.dtd">

<!-- nested_frames.html
     An example to illustrate nested frames
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Nested frames </title>
  </head>
  <frameset cols = "40%, *">
    <frameset rows = "50%, *">
      <frame src = "frame1.html" />
      <frame src = "frame2.html" />
    </frameset>
    <frameset rows = "20%, 35%, *">
      <frame src = "frame3.html" />
      <frame src = "frame4.html" />
      <frame src = "frame5.html" />
    </frameset>
  </frameset>
</html>
```

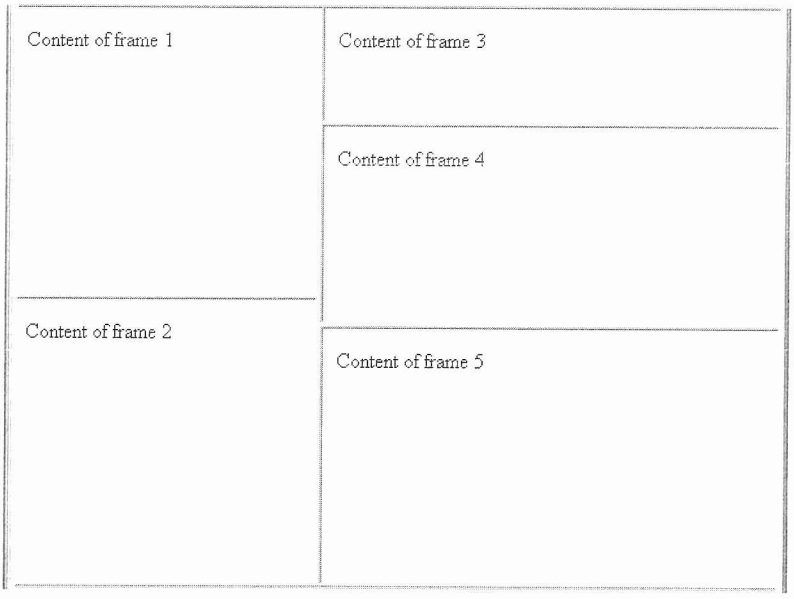


Figure 2.35 Display of `nested_frames.html`

2.11 Syntactic Differences between HTML and XHTML

There are some significant differences between the syntactic rules of HTML (or lack thereof) and those of XHTML. This section describes these differences.

Case sensitivity. In HTML, tag and attribute names are case insensitive, meaning that `<FORM>`, `<form>`, and `<Form>` are equivalent. In XHTML, all tag and attribute names must be in lowercase.

Closing tags. In HTML, closing tags may be omitted if the processing agent (usually a browser) can infer their presence. For example, in HTML, paragraph elements often do not have closing tags. The appearance of another opening paragraph tag is used to infer the closing tag on the previous paragraph. For example:

```
<p>
During Spring, flowers are born. ...
<p>
During Fall, flowers die. ...
```

In XHTML, all elements must have closing tags. For elements that do not include content, in which the closing tag appears to serve no purpose, a slash can be included at the end of the opening tag as an abbreviation of the closing tag. For example, the following two lines are equivalent:

```
<input type = "text"  name = "address" > </input>
```

and

```
<input type = "text"  name = "address" />
```

Recall that some browsers can be confused if the slash at the end is not preceded by a space.

Quoted attribute values. In HTML, attribute values must be quoted only if there are embedded special characters or whitespace characters. Numeric attribute values are rarely quoted in HTML. In XHTML, all attribute values must be double quoted, regardless of what characters are included in the value.

Explicit attribute values. In HTML, some attribute values are implicit; that is, they need not be explicitly stated. For example, if the `border` attribute appears in a `<table>` tag without a value, it specifies a default width border on the table. For example:

```
<table border>
```

This is illegal in XHTML, in which such an attribute is assigned a string of the name of the attribute. For example:

```
<table border = "border">
```

Other such attributes are `checked`, `multiple`, and `selected`.

`id` *and* `name` *attributes.* HTML markup often uses the `name` attribute for elements. This attribute was deprecated for some elements in HTML 4.0. The `id` attribute was added to nearly all elements with this same version of HTML. In XHTML, the use of `id` is encouraged, and the use of `name` is discouraged. In fact, the `name` attribute was removed for the anchor and map elements in XHTML 1.1. However, form elements must still use the name attribute because it is used in processing form data.

Element nesting. Although HTML has rules against improper nesting of elements, they are not enforced. Examples of nesting rules are: 1) an anchor element cannot contain another anchor element, and a form element cannot contain another form element; 2) if an element appears inside another element, the closing tag of the inner element must appear before the closing tag of the outer element; 3) block elements cannot be nested in inline elements; 4) text cannot be directly nested in body or form elements; and 5) list elements cannot be directly nested in list elements. In XHTML, these nesting rules are strictly enforced.

All of the XHTML syntactic rules are checked by the W3C validation software.

Summary

XHTML was derived from SGML. Without the style sheets described in Chapter 3, XHTML is capable of specifying only the general layout of documents, with few presentation details. The current version of XHTML is XHTML 1.1; it was released in 2001.

The tags of XHTML specify how content is to be arranged in a display by a browser (or other XHTML processor). Most tags consist of opening and closing tags to encapsulate the content that is to be affected by the tag. XHTML documents have two parts, the head and the body. The head describes some things about the document but does not include any content. The body includes the content and the tags and attributes to describe the layout of that content.

Line breaks in text are ignored by browsers. The browser fills lines in its display window and provides line breaks when needed. Line breaks can be specified with the `<br />` tag. Paragraph breaks can be specified with `<p>`. Headings can be created with the `<hx>` tags, where *x* can be any number from 1 to 6. The `<blockquote>` tag is used to set off a section of text. The `<sub>` and `<sup>` tags are used to create subscripts and superscripts, respectively. Horizontal lines can be specified with the `<hr />` tag.

Images in GIF, JPEG, or PNG format can be inserted into documents from files where they are stored with the `<img />` tag. The `alt` attribute of `<img />` is used to present a message to the user when his or her browser is unable (or unwilling) to present the associated image.

Links support hypertext by allowing a document to "point to" other documents, enabling the user to move easily from one document to another. The

target of a link can be a different part of the current document or the top or some other part of a different document.

XHTML supports both unordered lists, using the `<ul>` tag, and ordered lists, using the `<ol>` tag. Both of these kinds of lists use the `<li>` tag to define list elements. The `<dl>` tag is used to describe definition lists. The `<dt>` and `<dd>` tags are used to specify the terms and their definitions, respectively.

Tables are easy to create with XHTML, using a collection of tags designed for that purpose. `<table>` is used to create a table, `<tr>` is used to create table rows, `<th>` is used to create label cells, and `<td>` is used to create data cells in the table. The `colspan` and `rowspan` attributes, which can appear in both `<th>` and `<td>` tags, provide the means of creating multiple levels of column and row labels, respectively. The `align` and `valign` attributes of the `<tr>`, `<th>`, and `<td>` tags are used to tell the browser exactly where to put data or label values within their respective table cells. The `cellpadding` and `cellspacing` attributes are used to specify the distance between the content of a cell and its boundary and the distance between cells in a table, respectively.

XHTML forms are sections of documents that contain controls used to collect input from the user. The data specified in a form can be sent to a server-resident program in either of two methods, `get` or `post`. The most commonly used controls (text boxes, checkboxes, passwords, radio buttons, and the action buttons *Submit*, *Reset*, and *plain*) are specified with the `<input>` tag. The *Submit* button is used by the user to indicate that the form data is to be sent to the server for processing. The *Reset* button is used to clear all of the controls in a form. The text box control is used to collect one line of input from the user. Checkboxes are one or more buttons used by the user to select one or more elements of a list. Radio buttons are like checkboxes, except that within a collection, only one button can be on at a time. A password is a text box whose content is never displayed by the browser.

Menus are used to allow the user to select items from a list when the list is too long to use checkboxes or radio buttons. Menu controls are created with the `<select>` tag. A text area control, which is created with the `<textarea>` tag, creates a multiple line text-gathering box, with implicit scrolling in both directions.

In HTML and XHTML 1.0, the browser window can be divided into rectangles called *frames*, each of which can display a different document. The screen layout of frames is specified in a `<frameset>` tag, using the `rows` and `cols` attributes. One `<frame>` tag is given in the frameset content for each frame. The `src` attribute of `<frame>` gives the address of the document to be displayed in the frame. If a `<frame>` tag includes a `name` attribute, there can be a link in a different frame that refers to that name.

Review Questions

2.1 What does it mean for a tag or attribute of XHTML to be deprecated?

2.2 What is the form of an XHTML comment?

2.3 How does a browser treat line breaks in text to be displayed?

2.4 What is the difference in the effect of a paragraph tag and a break tag?

2.5 Which heading tags use fonts that are smaller than the normal text font size?

2.6 How do browsers usually set block quotations differently from normal text?

2.7 What does the `<code>` tag specify for its content?

2.8 What are the differences between the JPEG and GIF image formats?

2.9 What are the two required attributes of an `<img />` tag?

2.10 What is the purpose of the `alt` attribute of `<img />`?

2.11 What tag is used to define a link?

2.12 What attribute is required in all anchor tags?

2.13 Does XHTML allow nested links?

2.14 How is the target of a link usually identified in a case where the target is in the currently displayed document but not at its beginning?

2.15 What is the form of the value of the `href` attribute in an anchor tag when the target is a fragment of a document other than the one in which the link appears?

2.16 What is the default bullet form for the items in an unordered list?

2.17 What are the default sequence values for the items in an ordered list?

2.18 What tags are used to define the terms and their definitions in a definition list?

2.19 What is specified when the `border` attribute of a `<table>` tag is set to `"border"`?

2.20 What is the purpose of the `colspan` attribute of the `<th>` tag?

2.21 What is the purpose of the `rowspan` attribute of the `<td>` tag?

2.22 What are the `align` and `valign` attributes of the `<tr>`, `<th>`, and `<td>` tags used for?

2.23 What is the difference between the `cellspacing` and `cellpadding` attributes?

2.24 Describe the possible values of the `rows` attribute of `<frameset>`.

2.25 What does the `src` attribute of `<frame>` specify?

2.26 Of what use is a frame that does not initially have a document?

2.27 How is a frame specified in a link from a different frame?

2.28 What are controls?

2.29 Which controls discussed in this chapter are created with the `<input>` tag?

2.30 What is the default size of a text control's text box?

2.31 What is the difference between the `size` and `maxlength` attributes of `<input>` for text controls?

2.32 What is the difference in behavior between a group of checkbox buttons and a group of radio buttons?

2.33 Under what circumstances is a menu used instead of a radio button group?

2.34 What is the drawback of specifying the `multiple` attribute with a menu?

2.35 How are scroll bars specified for text-area controls?

Exercises

2.1 Create, test, and validate an XHTML document for yourself, including your name, address, and e-mail address. If you are a student, you must include your major and your grade level. If you work, you must include your employer, your employer's address, and your job title. This document must use several headings and `<em>`, `<strong>`, `<hr />`, `<p>`, and `<br />` tags.

2.2 Add pictures of yourself and at least one other image (your friend, spouse, or pet) to the document created for Exercise 2.1.

2.3 Add a second document to the document created for Exercise 2.1 that describes part of your background, using `background` as the link content. This document should have a few paragraphs of your personal or professional history.

2.4 Create, test, and validate an XHTML document to describe an unordered list of a typical grocery shopping list you write. (If you've never made up such a list, use your imagination.)

2.5 Create, test, and validate an XHTML document to describe an unordered list of at least four states. Each element of the list must have a nested list of at least three cities in the state.

2.6 Create, test, and validate an XHTML document to describe an ordered list of your five favorite movies.

2.7 Modify the list of Exercise 2.6 to add nested, unordered lists of at least two actors and/or actresses in your favorite movies.

2.8 Create, test, and validate an XHTML document to describe an ordered list with the following contents: The highest level should be the names

of your two parents, with your mother first. Under each parent, you must have a nested, ordered list with the brothers and sisters of your parents, in order with the eldest first. Each of the nested lists must have nested lists that list the children of your uncles and aunts (your cousins)—under the proper parents, of course. Regardless of how many aunts, uncles, and cousins you actually have, there must be at least three list items in each sublist below each of your parents and below each of your aunts and uncles.

2.9 Create, test, and validate an XHTML document to describe a table with the following contents: The columns of the table must have the headings Pine, Maple, Oak, and Fir. The rows must have the labels Average Height, Average Width, Typical Lifespan, and Leaf Type. You can make up the data cell values.

2.10 Modify, test, and validate an XHTML document from Exercise 2.9 to add a second-level column label, Tree, and a second-level row label, Characteristics.

2.11 Create, test, and validate an XHTML document that defines a table with columns for state, state bird, state flower, and state tree. There must be at least five states as rows in the table. You must include attribute specifications for `cellpadding` and `cellspacing`.

2.12 Create, test, and validate an XHTML document that defines a table that has two levels of column labels: an overall label, Meals, and three secondary labels, Breakfast, Lunch, and Dinner. There must be two levels of row labels: an overall label, Foods, and four secondary labels, Bread, Main Course, Vegetable, and Dessert. The cells of the table must contain a number of grams for each of the food categories.

2.13 Create, test, and validate an XHTML document that is the home page of a business, Tree Branches, Unlimited, that sells tree branches. This document must include images of and descriptions of at least three different kinds of tree branches. There must be at least one unordered list, one ordered list, and one table. Detailed descriptions of the different branches must be stored in separate documents that are accessible through links from the home document. You must invent several practical uses for tree branches and include sales pitches for them.

2.14 Create, test, and validate an XHTML document that includes two rows of frames with two frames in each row. The two left frames must occupy 25 percent of the width of the display. The two bottom frames must occupy 40 percent of the height of the display. The top left frame must display the name of your mother and all of her siblings. (There must be at least two siblings—make them up, if you must.) The bottom left frame must display the name of your father and all of his siblings. (Once again, there must be at least two siblings.) Each name in the left frames must be a link to a document that is displayed in the right frame when the link is